

The Kitchen Loop: User-Spec-Driven Development for a Self-Evolving Codebase

Yannick Roy
OxAgentKitchen@gmail.com

March 2026

Abstract

Code production is now a commodity; the bottleneck is *knowing what to build* and *proving it works*. We present the **Kitchen Loop**,¹ a framework for autonomous, self-evolving software built on a unified trust model: (1) a **specification surface** enumerating what the product claims to support; (2) “**As a User** $\times$ **1000**”, where an LLM agent exercises that surface as a synthetic power user at $\sim 1,000\times$ human cadence; (3) **Unbeatable Tests**, ground-truth verification the code author cannot fake; and (4) **Drift Control**, continuous quality measurement with automated pause gates. We validate across two production systems over 285+ iterations, producing 1,094+ merged pull requests with zero regressions detected by the regression oracle (methodology in Section 6.1). We observe emergent properties at scale: multi-iteration self-correction chains, autonomous infrastructure healing, and monotonically improving quality gates. The primitives are not new; our contribution is their composition into a production-tested system with the operational discipline that makes long-running autonomous evolution safe.

Keywords: Vibe Coding, AI Coding, Autonomous Software Engineering, LLM Agents, Dark Factory

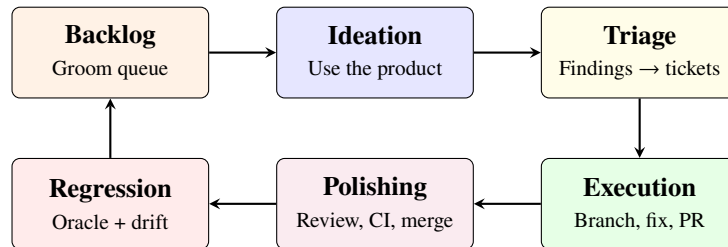


Figure 1: The Kitchen Loop: a six-phase autonomous improvement cycle.

¹Open-source: <https://github.com/Oxagentkitchen/kitchenloop>

1 Core Contributions

This paper makes four claims:

1. **“As a User” vs. task-completion.** Agentic systems should not just close tickets — they should systematically exercise a product’s specification surface the way a user would. The Kitchen Loop is a production-tested framework for this, grounded in synthetic user journeys rather than isolated issues.
2. **The unified trust model.** Autonomous evolution requires a unified trust model. Three components interlock to make it safe: a *specification surface* (what the product claims to do), *unbeatable tests* (ground-truth verification the author can’t fake), and a regression oracle with *drift control* and automatic pause gates.
3. **Unbeatable tests.** Correctness requires adversarial, multi-model review. Implementer-written tests are necessary but insufficient — in our deployments, 38 passing unit tests coexisted with complete feature failure. The Kitchen Loop enforces correctness through adversarial UAT gates (sealed test cards, fresh evaluator, zero context) and mandatory cross-model review: every PR is challenged by independent agents (Codex, Gemini, CodeRabbit) before merge. No output is accepted as-is from the model that wrote it.
4. **Bounded production evidence.** Across two production systems and 285+ iterations, the Kitchen Loop produced 1,094+ merged pull requests with zero regressions detected by the regression oracle (methodology in Section 4.1), monotonically improving quality gates (76–91% → 100%), and a cost of ~\$0.38 per merged PR.

Key definitions used throughout:

Term	Definition
Specification surface	The enumerable set of capabilities a product claims to support — the input to coverage-exhaustion
Unbeatable test	A test that verifies outcomes against ground truth that the code author cannot fake
Regression oracle	A repeatable, bounded test that answers “is the system at least as good as before?”
Coverage-exhaustion mode	An operating regime where the agent systematically exercises every combination in the specification surface until coverage gaps approach zero
UAT gate	Adversarial user-acceptance testing by a fresh evaluator with zero implementation context
Drift control	Continuous measurement of quality trends with automated pause gates that halt the loop when metrics degrade

2 Executive Summary (TL;DR)

The method

An LLM agent uses the product as a synthetic power user at $\sim 1,000\times$ human cadence against its specification surface and beyond, validates through unbeatable tests, and controls drift and regression before accepting the work to ensure the product self-evolves in the right direction (Figure 2).

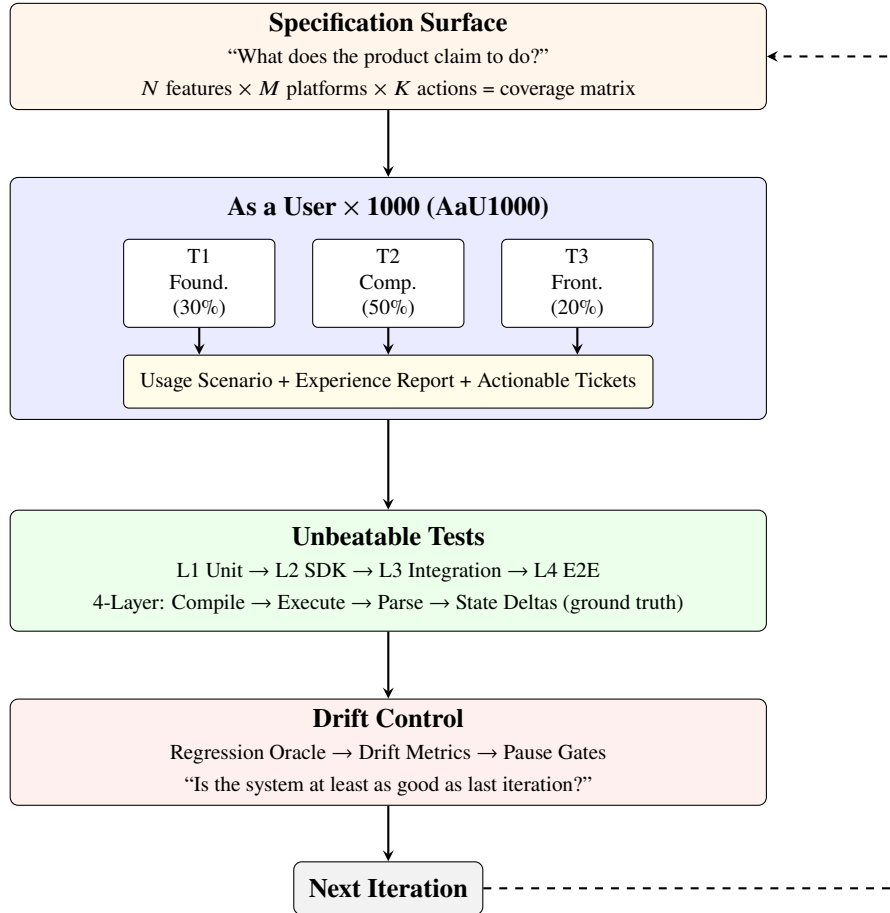


Figure 2: The unified trust model: each iteration flows through the full verification stack before proceeding.

The architecture

A six-phase loop (Figure 1) with automated drift control and pause gates.

The results

Seven weeks, two production systems:

Kitchen Loop Results

Iterations = 285+	PRs merged = 1,094+	Tickets = 700+
Tests = 13,000+	Regressions = 0	Cost/PR = ~\$0.38
Quality gates	L1 100%, L2 100%, L3 100% (from 76–91%)	
Canary escapes (Tier 1)	0 across all 163 signal iterations	
Monthly cost	~\$350 (flat-rate subscriptions, both systems)	
Iteration speed	5 min (signals) to 80–230 min (execution)	
Production incidents	0	

3 The Post-Commodity Code Thesis

3.1 Code Is No Longer the Hard Part

LLM-based coding agents can produce functional code at a rate that renders the writing step non-limiting. Robbes et al. (2026) find that 15–23% of mature open-source projects adopted coding agents within just nine months of tool availability [4]. A senior engineer’s value is shifting from writing code to knowing *which* code to write, *why*, and *how to prove it’s correct*.

Yet the productivity story is more nuanced than adoption rates suggest. Becker et al. (2025) conducted the most rigorous randomized controlled trial to date: experienced open-source developers using AI copilots were 19% *slower* than those working without AI, contradicting both developer self-estimates (+20% speedup) and expert forecasts (+38% speedup) [12]. He et al. (2025) provide causal evidence that Cursor adoption produces a large but transient velocity boost (+281% lines added in month one, dissipating by month three) accompanied by persistent quality degradation: +30% static analysis warnings, +42% cognitive complexity, and +7% code duplication [11]. The accumulated complexity feeds back into velocity — every doubling of complexity reduces future velocity by ~64.5% [11]. The Kitchen Loop can be read as a response to both findings: by shifting emphasis from code generation to specification and unbeatable verification, it addresses the quality degradation that erodes velocity gains and the lack of structural verification that leaves AI-assisted development slower than expected.

The same shift is happening to code review. Automated review tools can identify bugs, style violations, and security issues at a pace no human reviewer can match. The human reviewer’s role shifts from “find the bug” to “judge whether this code serves the product’s intent.”

This creates a new bottleneck: **specification and verification**. The hard problems are now:

- What should the product do? (Specification)
- Does the product actually do it? (Verification)
- Is the product getting better or worse over time? (Drift control)

3.2 The Turing Test for Code

Alan Turing’s insight was that you don’t need to understand how a machine thinks — you need a test rigorous enough that passing it constitutes sufficient evidence of capability. The machine can remain a black box. The test is the arbiter.

We apply the same principle to AI-generated code. When an LLM agent writes a function, a connector, or a full feature, you face a choice:

- **Option A:** Read every line, understand every branch, verify the logic manually. This scales poorly when the agent produces thousands of lines per day.

- **Option B:** Build tests so rigorous that if the code passes them, you can *trust* it works — without needing to audit every line. The code becomes a black box. The tests are the Turing Test.

Empirical evidence supports both options coexisting in practice. Huang et al. (2025) find that 69% of professional developers carefully review every agentic change and 75% read every line of AI-generated code — but crucially, developers working on *unfamiliar* tasks let agents drive implementation while monitoring program *outputs* rather than reviewing code line-by-line [2]. This validates Option B for the regime where the Kitchen Loop operates: autonomous agents on structured tasks with rigorous output verification.

The urgency for Option B is underscored by a growing QA crisis in AI-assisted development. Fawzy et al. (2025) find that 36% of practitioners using AI code generation skip quality assurance entirely, 18% place uncritical trust in AI output, and 10% delegate QA back to the same AI that wrote the code [1]. The result is predictable: 68% of practitioners characterize the output as “fast but flawed” [1]. Voluntary QA discipline is demonstrably insufficient; structural enforcement — tests that *cannot* be skipped — is the only reliable solution.

Option B requires a specific kind of test: not unit tests written by the same agent that wrote the code (the agent could pass its own tests trivially), but *end-to-end verification against real-world state*. For a DeFi SDK, this means executing transactions on chain forks and verifying balance changes. For a signal platform, this means cross-referencing claims against live data APIs. For a web application, this means browser automation against a real rendering engine. For any product, it means testing at the boundary where the software meets reality.

We call these **unbeatable tests** — tests that verify outcomes against ground truth that the code author cannot fake. The test doesn’t care *how* the code achieves the result — only that it does.

It is of utmost importance to be aware of the weaknesses of LLMs when it comes to tests. There is a fallacy to believe that LLMs are so good at writing code, that they should be equally good at writing tests. They aren’t. They often obsess over the test itself forgetting that passing the test is not the goal in itself. That obsession to pass the test often leads to cheating, modifying the test or even writing side scripts nailing the test, leaving the actual codebase untested.

4 Related Work

The Kitchen Loop exists within a rapidly evolving landscape of agentic software engineering. We position our contributions against four categories of related work.

4.1 Autonomous Coding Agents

SWE-agent (Yang et al., 2024) and **AutoCodeRover** (Zhang et al., 2024) demonstrated that LLM agents can resolve GitHub issues autonomously by navigating codebases, editing files, and running tests. **OpenHands** (Wang et al., 2024) and **Devin** (Cognition, 2024) extended this to multi-step workflows including environment setup, debugging, and deployment. **RepairAgent** (Bouzenia et al., 2024) focused specifically on automated program repair with LLM-driven fault localization.

These systems operate in **task-completion mode**: given an issue, produce a patch. The Kitchen Loop operates in **coverage-exhaustion mode**: given a specification surface, systematically exercise every combination until the gap between spec and reality approaches zero. The unit of work is not “resolve this issue” but “attempt this user scenario end-to-end and document everything that breaks.” Task completion is a component of the loop (the Execute phase), not the loop itself.

4.2 Self-Improving and Self-Evolving Agents

SICA [15] introduced agents that edit their own prompts and tools based on execution feedback, achieving self-improvement without human intervention. AlphaEvolve [16] demonstrated repository-scale code

evolution using LLMs guided by automated evaluators. Confucius SDK [17] proposed a meta-agent build-test-improve loop for SDK development.

The Kitchen Loop shares the self-improvement property (Section 13) — it discovers its own infrastructure bugs and updates its own skill files. The key difference is that self-improvement in the Kitchen Loop is *anchored to a specification surface and regression oracle*, not to an optimization objective. The loop doesn’t optimize for a metric; it converges toward a specification. This prevents the “goodharting” failure mode where an agent optimizes a proxy metric while the product degrades in unmeasured dimensions.

4.3 Long-Running and Looping Patterns

Ralph Wiggum loops (Huntley, 2025; snarktank/ralph) demonstrated that autonomous agents can produce 1,000+ commits overnight in a continuous loop. Multi-agent SE frameworks like **MetaGPT** (Hong et al., 2024), **ChatDev** (Qian et al., 2024), and **ALMAS** assigned different agent roles (PM, architect, coder, tester) to simulate a software team.

The Kitchen Loop draws from this lineage but adds three structural elements that Ralph-style loops lack: (1) a **production-parity ticket management layer** (Linear) where human and AI tickets are treated identically, preventing the loop from diverging from human priorities; (2) **automated drift control** with pause gates that halt the loop when quality degrades; and (3) the **three-tier strategy model** (Foundation/-Composition/Frontier) that ensures balanced coverage rather than random or greedy scenario selection.

4.4 Spec-Driven and Verification-Focused Approaches

SpecRover (Ruan et al., 2024) used natural-language specifications to guide automated debugging. **AgileGen** and **Augmented Agile** explored LLM-assisted requirements engineering and test generation from user stories. **USEagent** proposed user-story-driven end-to-end testing.

The Kitchen Loop’s AaU1000 method is closest to USEagent’s vision but differs in scope and trust model. USEagent generates tests from user stories; AaU1000 generates *usage scenarios* from specification surfaces and validates them through *unbeatable tests* (4-layer verification against ground truth). The distinction matters: a test generated from a user story proves the story is implemented; a usage scenario validated through state-delta verification proves the product *actually works* in the way a real user would experience it.

4.5 Vibe Coding and the QA Crisis

Fawzy et al. (2025) find a “speed-quality paradox” across 101 practitioner sources: 62% of vibe coders cite speed as their primary motivation, yet 68% characterize output as “fast but flawed” [1]. Huang et al. (2025) confirm that professional developers reject the vibe coding paradigm, carefully controlling agents through strategic plans — averaging only 2.1 execution steps per prompt despite plans spanning 70+ steps [2]. Their task suitability taxonomy identifies agents as unsuitable for business logic, domain knowledge, complex reasoning, and security-critical code — precisely the categories where the Kitchen Loop’s spec surface and UAT gate provide structural guardrails.

4.6 Benchmark Contamination, Adoption, and Quality Evidence

The dominant evaluation benchmark, SWE-Bench, suffers from data contamination: Liang et al. (2025) show a 23-percentage-point accuracy gap between memorized and novel repositories [6], while Thai et al. (2025) find GPT-5 scores 65% on single-issue tasks but only 21% on multi-file evolution tasks [7]. Meanwhile, adoption is accelerating — Robbes et al. (2026) find 15–23% of mature open-source projects adopted coding agents within nine months [4] — but quality lags: 90.6% of agent-authored PRs receive zero human review [3], and ~40% of Copilot-generated code contains security vulnerabilities [10]. The

Kitchen Loop addresses both problems: it rejects benchmark-style evaluation in favor of verification against live product state, and interposes adversarial verification between generation and merge.

4.7 Multi-Agent Design Patterns

Cai et al. (2025) systematically review 94 LLM-based multi-agent systems for software engineering, identifying 16 design patterns ranked by adoption frequency [8]. The Kitchen Loop composes seven of these patterns into a single integrated loop: Role-Based Cooperation (six-phase loop with distinct roles per phase), Self-Reflection (Polish and Regress phases), Cross-Reflection (UAT Gate), Debate-Based Cooperation (Discussion Manager), Voting-Based Cooperation (multi-model tribunal), Tool-Agent Registry (skills directory), and Agent Evaluator (regression oracle). Notably, three of these — Debate-Based Cooperation (4.3% adoption), Voting-Based Cooperation (3.2%), and Agent Evaluator (3.2%) — are among the least adopted patterns in the literature [8], suggesting the Kitchen Loop operationalizes architectural ideas the field has not yet widely implemented.

4.8 Our Differentiation

Table 2: Comparison of agentic software engineering approaches.

	Task-Completion	Ralph Loops	Self-Improving	Kitchen Loop
Unit of work	Issue → Patch	Commit	Optimization step	Scenario → experience report
Stopping cond.	Issue resolved	Time/count	Metric plateau	Specification exhausted
Coverage	Reactive (given issues)	Random/greedy	Objective-guided	Three-tier (F/C/F)
Quality gate	CI passes	CI + typecheck	Evaluator function	4-layer + multi-model tribunal
Drift control	None	Basic (progress log)	Implicit (metrics)	Explicit (oracle + pause gates)
Human role	PR review	Manual oversight	None	Shared backlog, ticket parity
Self-improvement	None	Manual (AGENTS.md)	Core feature	Anchored to spec + oracle

From our analysis, three distinct operating regimes for agentic software engineering emerge:

Table 3: Three operating regimes for agentic software engineering.

Regime	Unit of Work	Stopping Cond.	Verification Target	Failure Mode
Task-completion	Issue	Issue resolved	Patch correctness	Local fix / global mismatch
Metric-optimization	Objective step	Metric plateau	Proxy metric	Goodharting
Coverage-exhaustion	User scenario	Surface exhaustion	User-visible behavior	Drift / incomplete surface

The Kitchen Loop operates in the coverage-exhaustion regime. The regime determines the system’s failure mode: task-completion risks local fixes that break global behavior; metric-optimization risks Goodharting on proxy measures; coverage-exhaustion risks drift or incomplete specification surfaces, which the drift control mechanisms in Section 7 are designed to address.

5 The “As a User x 1000” Method (AaU1000)

5.1 The Core Insight

The most reliable signal about whether a feature works is a real usage attempt — not a unit test, not a code review, but someone exercising the product end-to-end. The problem: human usage attempts are slow and expensive. The AaU1000 method replaces the “someone” with an LLM agent that:

1. **Selects a realistic usage scenario** derived from the product’s specification surface

2. **Attempts it as a real user would** — writing code, running it, observing what breaks
3. **Documents failures as actionable tickets** — not vague observations but precise bug reports with reproduction steps, root cause hypotheses, and file-level pointers
4. **Fixes those failures immediately** — implementing the fix, writing tests, and shipping a PR
5. **Watches for regression and drift** — verifying nothing else broke and the codebase is not getting worse

And then does it again. At whatever cadence the infrastructure allows.

5.2 Quantifying the 1000x

The “1000x” is an order-of-magnitude claim, not a precise multiplier. We ground it empirically:

Single-thread velocity: A senior engineer realistically ships ~15-25 merged PRs per month (bug fixes, features, tests). Each Kitchen Loop instance runs single-threaded per product. The strategy framework produced 728+ merged PRs in ~5 weeks (~145/week); the signal platform produced 366 in 17 days (~150/week). Per-system, this is a **~24-48x single-thread throughput increase** depending on domain iteration speed (the signal platform iterates ~25x faster than the SDK). Combined across both systems running concurrently, total output was 1,094+ merged PRs.

Scenario throughput: The DeFi strategy framework’s loop completes one full usage scenario (ideate → implement → test → fix → regress) in 80-230 minutes. A human attempting the same scenario takes 1-3 days. This is a **7-25x per-scenario speedup**.

Parallelization potential: The loop currently runs single-threaded per product. Running N loops on N products (as demonstrated with SDK + Edge concurrently) scales linearly. With parallel execution infrastructure (dynamic port allocation, containerized test environments), a single product could run N loops exploring different specification regions simultaneously. The 1000x represents the *achievable ceiling* with moderate parallelization — not the current single-thread reality.

5.3 Spec-Driven, Not Spec-Speculative

The method is scoped to **what exists and is expected to work today** — not speculative feature engineering, benchmarking, or user research.

5.4 The “Obviously Missing” Signal

The most important class of findings is the **“obviously missing” signal**: features any competent user would expect, but that don’t work. These are objectively verifiable failures — a function returning `AttributeError`, a missing configuration entry, a gas cap blocking all transactions on a chain. The loop discovers verifiable failures, not speculative issues.

5.5 The Three-Tier Strategy Model

The Kitchen Loop formalizes a **three-tier scenario generation model** that ensures balanced coverage across the product’s maturity spectrum:

T1	30%	Foundation — “Does the basic stuff work perfectly?”
T2	50%	Composition — “What breaks when we combine things?”
T3	20%	Frontier — “What’s missing for the next generation?”

Tier 1 — Foundation (30%) exercises a single feature on a single platform or configuration. One integration, one action, happy path. These scenarios should be trivially achievable by a new user in a few minutes. If anything goes wrong, it is a critical regression. Foundation iterations maintain the baseline: the easy stuff must always be bulletproof.

Tier 2 — Composition (50%) combines two or more features in creative ways. Multi-service flows, indicator-driven behavior, multi-step workflows, cross-platform configuration stress tests. The goal is to find bugs at the *seams* between components that pass individually but fail in combination. This is where the majority of novel bug discovery happens, because the combinatorial space of feature pairs and triples is far larger than the space of individual features.

Tier 3 — Frontier (20%) deliberately reaches beyond the product’s current capabilities. The deliverable shifts from “working scenario” to “gap analysis”: what would you need to build this? What’s missing? What would it unlock? The experience report emphasizes specific missing features, ordered by implementation effort and user value.

5.6 The Self-Expanding Property

The three tiers form a **self-reinforcing growth cycle**. This is the “secret sauce” that transforms a random agent into a strategic engineer:

```
Iteration N:   Features = {A, B}
  T1 (Foundation):  test A, test B                      (2 scenarios)
  T2 (Composition): test A+B, test B+A                  (2 scenarios)
  T3 (Frontier):    ``I need C to do A+C'' → gap analysis (1 gap report)
--- C gets built ---
Iteration N+k:  Features = {A, B, C}
  T1 (Foundation):  test A, test B, test C              (3 scenarios)
  T2 (Composition): test A+B, A+C, B+C, A+B+C          (4+ scenarios)
  T3 (Frontier):    ``Now I want D to do A+C+D''        (1 gap report)
```

Foundation grows linearly with each new feature. Composition grows **superlinearly** — each new feature can be combined with every existing feature and pair. Frontier pushes the boundary outward. The loop evolves the product by systematically testing what exists and identifying the next most valuable capability.

Design Principles

P1. Ground-Truth Verification

Trust in AI-generated code should be proportional to how ground-truth-verifiable the test outcomes are, not to test coverage percentage.

P2. Weakest-Evaluator

A user test is only trustworthy if a minimally capable evaluator can execute it. (§4.8)

P3. Spec-Anchored Improvement

Self-improving agents should optimize toward specification satisfaction, not proxy metrics. (§2.2, §3.5)

P4. Drift-Before-Failure

Autonomous loops need continuous trend monitoring, not only binary quality gates. (§5.5)

6 Unbeatable Tests: The Multi-Tier QA Framework

6.1 Methodology: What “Zero Regressions” Means

Throughout this paper, we report “zero regressions introduced by loop-merged code.” This claim requires precise definition to be falsifiable:

- **Detection method:** Every iteration ends with an automated regression oracle run (Section 5.2). For the DeFi strategy framework, this means executing demo strategies on Anvil chain forks and verifying 4-layer state deltas. For the signal platform, this means running the full quality-gate pipeline (structural, factual, temporal, cognitive) plus anti-signal canaries.
- **Measurement window:** Continuous — every iteration, not sampled. The oracle runs after every merge, not on a periodic schedule.
- **Scope:** The claim covers regressions *detected by the oracle* in code *merged through the loop’s own process*. It does not claim the codebase is bug-free, nor that latent regressions undetectable by the oracle do not exist. The oracle’s coverage is bounded by its test suite (10,913 unit tests, 62 demo strategies, 77 signal verifiers).
- **Exclusions:** Environmental failures (API timeouts, chain fork instability) are distinguished from code regressions by correlation with recent merges. A failure that reproduces on the pre-merge commit is environmental; a failure introduced by a specific PR is a regression.

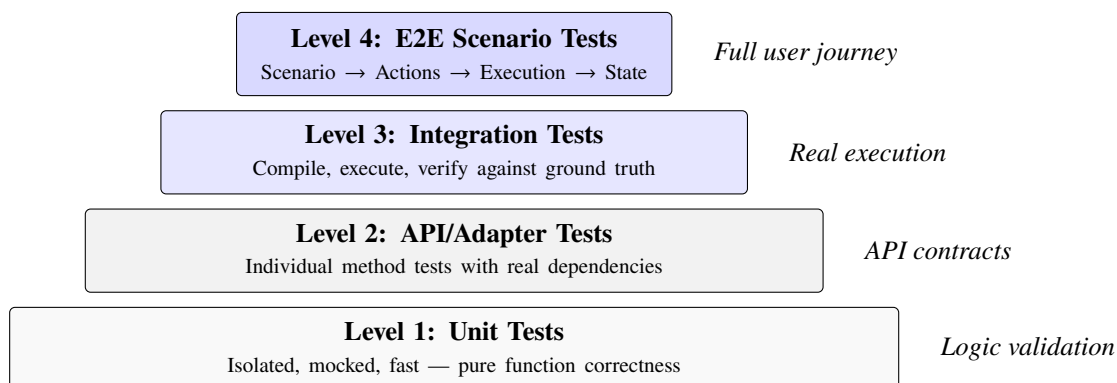
This definition makes the claim auditable: any third party with access to the oracle suite and git history can reproduce the measurement.

6.2 Why Tests Are the New Competitive Advantage

Gao et al. (2025) find functional bugs in 78% of studies on AI-generated code, with ~40% of Copilot code containing security vulnerabilities [10]. Dominant benchmarks (HumanEval, MBPP) rely on Pass@k metrics that overlook semantic correctness [10], and Liang et al. (2025) show SWE-Bench performance is inflated by data contamination (76% accuracy on memorized paths vs. 53% on novel repositories) [6]. Unbeatable tests sidestep both failure modes: they verify against live product state that changes with every deployment, making both contamination and pass-rate gaming irrelevant.

This requires tests at multiple tiers, each catching a different class of defect that lower tiers miss.

6.3 The 4-Level Testing Pyramid



The critical insight: L1 and L2 are *necessary but not sufficient*. A function can pass all its unit tests and still fail in production because the unit tests don’t exercise the real execution environment. **L3 and L4 are the unbeatable tests** — they verify against ground truth (real state, real APIs, real execution) that the code author cannot fake.

Level	What It Tests	Speed	Trust Level	Written By
L1	Isolated logic, calculations	Fast (ms)	Low — proves logic, not integration	Agent or human
L2	API methods, adapters	Medium (s)	Medium — proves API contracts	Agent or human
L3	Full execution pipeline	Slow (min)	High — proves real-world behavior	Loop + agent
L4	Complete user journeys	Slowest	Highest — proves product works	Loop (AaU1000)

6.4 The 4-Layer Verification Pattern

Every L3 integration test implements **four verification layers**, each catching a different class of defect. We illustrate with the DeFi strategy framework (Case Study A, Section 10); other domains substitute their own verification layers (e.g., browser automation for web apps, API contract testing for backends — see Section 16 for adaptation examples).

Layer	Name	What It Catches	How
1	Compilation	Wrong params, missing config, type errors	Build/compile the action, assert success
2	Execution	Runtime failures, permission errors, timeouts	Execute against real env, assert success
3	Output Parsing	Wrong decoding, missing fields, malformed data	Parse output, assert expected data extracted
4	State Deltas	Silent failures, partial exec, wrong outcomes	Measure state before/after, assert exact deltas

A test that only compiles is **incomplete**. A test that compiles and executes but doesn't check state deltas is **dangerously incomplete** — it could silently succeed while doing the wrong thing. Layer 4 is what makes the test unbeatable: it verifies the *outcome*, not just the *execution*.

For failure-mode tests: 3 layers are required (compilation, execution, state deltas), and state conservation **MUST** be asserted (state before and after remain unchanged). The system must not silently lose assets when it fails.

6.5 The Coverage Matrix: Exhaustive by Design

The specification defines a coverage matrix: every combination of feature, platform, and action type that the product claims to support. The test suite's goal is to fill every cell in this matrix.

For a product with N features, M platforms, and K action types, the matrix has N x M x K cells. Each cell represents a claim: "Feature X works on Platform Y for Action Z." Each empty cell is an untested claim — a place where the product might silently fail.

The Kitchen Loop fills these cells systematically, prioritizing by risk:

1. **P0**: Core features on primary platforms (table stakes)
2. **P1**: Core features on secondary platforms (breadth)
3. **P2**: Advanced features on primary platforms (depth)
4. **P3**: Edge cases and aggregators (completeness)

No manual QA process can fill a matrix with hundreds or thousands of cells on a weekly cadence. The Kitchen Loop can, because it generates and runs tests at AI speed.

6.6 Anti-Signal Canaries: Testing the Tests

For systems where outputs are non-deterministic (signals, recommendations, generated content), the Kitchen Loop uses **anti-signal canaries**: intentionally crafted bad inputs injected alongside real ones to verify the quality gate catches what it should.

Four tiers of increasing deceptiveness:

Tier	Name	Description	Expected	Observed (iter 163)
1	Obviously Bad	Glaring structural or factual errors any gate should catch	100%	100% (0 escapes / 163 iters)
2	Shadow	Factually true but stale, low-novelty, or below threshold	50–80%	100% (from 33% at iter 1, fixed iter 124)
3	Adversarial	Real data with wrong conclusions — fools deterministic and LLM checks	30–60%	100% (from 67% at iter 1)
4	Mixed T/F	Blend of valid and fabricated data in one signal — partial-failure detection	20–50%	100% (added iter ~100, fixed iter 124)

Additionally, **3 API degradation canaries** test resilience to partial external API failures (e.g., a data source returning errors or timeouts). These verify the quality gates degrade gracefully rather than producing false passes when dependencies fail.

The canary system provides a **known-bad baseline** that the regression phase can measure against. Tier 1 canary escapes are treated as a critical warning signal for operator review. The monotonic improvement across all tiers — from partial catch rates in early iterations to 100% across all 4 tiers by iteration 124 — demonstrates that the loop’s quality infrastructure improves alongside the product it protects.

A notable finding from the Edge deployment: Tier 2 catch rates were stuck at 33% for 70+ iterations. Early attempts to improve them via the L4 LLM tribunal (GPT/Claude/Gemini judgment) produced non-deterministic results — catching 0-2 canaries per iteration with no convergence. The durable fix came from encoding failure patterns as *deterministic rules* (e.g., STALE_NARRATIVES, STALE_CATALYSTS), achieving 100% catch rate that held for 40+ subsequent iterations. This validates Design Principle P1 (Ground-Truth Verification): for safety-critical gates, deterministic verification outperforms probabilistic LLM judgment.

6.7 Multi-Model Review Tribunals

For decisions that require judgment (architectural choices, ambiguous test results, code quality assessment), the Kitchen Loop uses **multi-model tribunals**: three independent AI reviewers evaluate the same artifact in parallel. Findings are synthesized with consensus classification:

- **Consensus** (all three agree): treated as a confirmed finding
- **Majority** (two agree): treated as a likely finding, prioritized for action
- **Solo** (one reviewer): flagged for human judgment

This reduces the false-positive rate of any single model and provides higher-confidence assessments for critical decisions. Section 7 extends this concept into a full structured deliberation system — the Discussion Manager — with multi-round debate, epistemic safeguards against sycophancy, and empirical validation across 23 production discussions.

6.8 The Adversarial UAT Gate: “How Would a User Test This?”

Unit tests prove code works; they don’t prove features work for users. When the same agent that implements a feature also tests it, three failure modes emerge:

1. **Happy-path blindness** — the implementer only tests the case they built for
2. **Context leakage** — the agent “knows” the implementation and unconsciously compensates for gaps
3. **Cheating** — AI models optimize for green checks, not product truth. They write side scripts, mock data, and reinterpret assertions to force a pass

The solution is not “ask the model to test honestly” but to **design the loop so honesty is the easiest behavior and cheating is mechanically visible**. The UAT Gate implements this through a three-step adversarial protocol:

6.9 Step 1: “How would a user test this?”

After implementing a ticket and creating a PR, the implementing agent must write a **sealed test card** — a step-by-step recipe that any user could follow to verify the feature works. The test card format is strict:

- Every step has an **exact command** (no placeholders requiring judgment)
- Every step has an **exact expected exit code** and **exact output assertions** (not “should work” — specific strings)
- At least one step verifies that **bad input is rejected** (not just happy path)
- **No manual code edits** — if testing requires configuration, the implementer ships a fixture in the PR
- **No implementation details** — user-visible behavior only

This is the “as a user, how can I test this?” forcing function. If the implementer cannot write a card that demonstrates the feature working end-to-end, that’s a signal the feature isn’t done — even if all unit tests pass.

6.10 Step 2: “Clear session — remove biases”

A fresh agent is spawned with **zero implementation context** to execute the test card:

- **Information wall:** The evaluator receives only the test card. No diff, no ticket, no code context, no conversation history from the implementing agent.
- **Weakest model:** The evaluator uses the weakest available model (e.g., Haiku) as a “dumb user” proxy. A strong model compensates for bad test cards by “figuring out” what the feature does. A weak model fails if the card is incomplete — which is exactly the signal you want. If the weakest model can follow the card and get expected results, a real user can too.
- **Isolated worktree:** The evaluator runs in a clean git worktree — a separate copy of the repository with pristine state.
- **Read-only mandate:** The evaluator is explicitly forbidden from editing any product file. Its prime directive is to *disconfirm* the feature, not confirm it.

6.11 Step 3: “Do it — show me the results”

The evaluator executes every step and produces structured evidence: raw command output, exact exit codes, actual-vs-expected comparisons. Then the implementing agent performs a **mechanical integrity check**:

Three-Layer Anti-Cheating

1. **Information wall.** Evaluator gets ONLY the test card. No diff, no ticket, no implementation context.
2. **Mechanical integrity check.** After evaluation: `git diff` on the UAT worktree. Any product file modification = `EVAL_CHEAT_FAIL`. Any untracked files outside evidence dir = `FAIL`.
3. **Evidence structure.** Every step: raw command output + exit codes. Missing steps = failure. “It worked” without output = failure.

The verdict taxonomy makes the outcome actionable:

Verdict	Meaning	Action
PASS	Feature works from user perspective	Proceed, attach evidence to PR
PRODUCT_FAIL	Feature is broken	Keep ticket open, tag PR <code>uat-failed</code>
UAT_SPEC_FAIL	Test card is ambiguous or un-runnable	Log for process improvement, don't block
EVAL_CHEAT_FAIL	Evaluator modified product files	Serious process issue, flag for human review

6.12 Real-World Example: The Backtest Service Gap

This example illustrates exactly why the UAT gate exists.

An agent implemented a backtest HTTP service with **38 passing unit tests** covering HTTP routing, job lifecycle state machines, model serialization, and capacity limits. All tests green. Lint passes. PR created. Ticket moved to “In Review.”

But nobody ever actually started the service and sent a request to it.

The real smoke test — `curl -X POST http://localhost:8000/api/v1/backtest` followed by polling for completion — would have revealed that `PnLBacktester()` was instantiated with no constructor arguments. The data providers, fee models, and slippage models were never wired up. The service accepted jobs and immediately failed them.

38 unit tests passed. The feature was completely broken.

With the UAT gate, the implementer must write a test card whose steps include starting the service, submitting a job, polling for completion, and verifying results contain data. The Haiku evaluator runs this card blindly. Step 3 returns `"status": "failed"`. Verdict: **PRODUCT_FAIL**. The ticket stays open until the agent actually wires up the backtest pipeline.

The implementer can't dodge this: - Can't write “run the unit tests” as the card — validation rejects it (not user testing) - Can't test only HTTP routing — the card rules require testing the actual user journey - Can't weaken assertions — “Expected output contains: completed” is binary - Can't skip the gate — mandatory for any change to product code. In autonomous loops where 90.6% of agent-authored PRs receive zero human review [3], the UAT gate fills this gap mechanically.

7 Controlling Regression and Drift

7.1 The Drift Problem

A self-evolving codebase faces a unique risk: **quality drift**. Each iteration produces code that passes its own tests — but does the accumulation of changes make the system better or worse? Without continuous measurement, the answer is unknowable until a user hits a regression in production.

Shukla et al. (2025) quantify this risk: iterative LLM code refinement paradoxically *degrades* security, with vulnerabilities rising from 2.1 per sample in early iterations to 6.2 by iterations 8–10 — a 37.6% increase in critical vulnerabilities after just five iterations [9]. He et al. (2025) corroborate the drift risk from a different angle: Cursor adoption produces persistent quality degradation (+30% static analysis warnings, +42% cognitive complexity) that outlasts the transient velocity gains [11].

The Kitchen Loop treats regression control as a first-class concern, not an afterthought. Every iteration ends with a regression phase that answers: “Is the system at least as good as it was before this iteration?”

7.2 The Regression Oracle

Each product domain requires a **regression oracle** — a repeatable test that answers “is the system still working?” in bounded time. The oracle’s properties:

- **Deterministic:** Same inputs produce same pass/fail on same codebase
- **Comprehensive:** Covers the product’s critical paths
- **Fast enough:** Must complete within the iteration budget
- **Independent of the loop:** The oracle tests the product, not the loop’s own output

Mode	Duration	Coverage	When to Use
Full	120-150 min	All scenarios, all platforms	Scheduled weekly
Quick	30-40 min	One scenario per platform	Every iteration

7.3 The Blocked Combos Registry

A critical operational tool is the **Blocked Combos** registry — a machine-readable list of feature/platform combinations that are known-broken and should not be re-tested by the ideation phase. This prevents the loop from wasting iterations on known-broken paths.

When a blocking ticket is resolved, the combo is removed and becomes available for ideation again. The registry grows and shrinks as bugs are found and fixed, creating a living map of the product’s actual (not assumed) capability surface.

7.4 Drift Metrics

The loop tracks several metrics across iterations to detect drift:

Metric	Healthy Trend	Warning Signal
Test count	Growing or stable	Declining
Pass rate	Stable at >95%	Declining over 3+ iterations
Bug discovery rate	Declining (maturity)	Sudden spike (regression)
Oracle pass rate	100%	Any failure correlated with recent changes
Blocked combos	Declining	Growing without corresponding fix tickets
Canary escape rate	0% for Tier 1	Any Tier 1 escape

The Regress phase verifies iteration history completeness before drift analysis; missing rows are back-filled automatically, since gaps would silently defeat sliding-window trend detection. A complementary cross-PR interaction detector identifies structural regression patterns — files added in one PR and deleted by another, revert commits, and high-churn files modified by 3+ independent merges — that the regression oracle cannot catch because they span multiple PRs.

7.5 Automated Pause Gates

Five automated gates determine whether the loop should continue or pause for human review:

Drain Mode: When open PRs exceed a threshold (default: 10), the loop automatically enters drain mode — skipping all phases except Polish and increasing the PR processing limit. When PRs drop below the exit threshold (default: 5), normal operation resumes. The original phase configuration is saved and

Gate	Trigger	Response	Auto?
Regression Failure	Oracle pass rate drops below threshold	Pause after N consecutive failures (default 3)	Semi
Canary Escape	Tier 1 canary passes quality gate	Warn operator	Advisory
Drift Threshold	Quality metric declines 3+ consecutive iters	Warn operator	Advisory
Backpressure	Open PRs exceed threshold	Enter drain mode (polish-only)	Yes
Starvation	Execute starved N consecutive iters	Monitor-only, alert human	Yes

restored, so drain mode is transparent to the running loop. This prevents the failure mode where the loop produces PRs faster than it can merge them, causing an unbounded backlog.

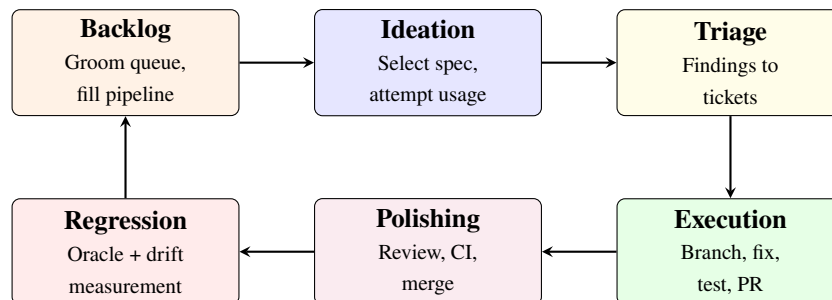
Starvation Gate: When the Execute phase produces zero output for N consecutive iterations (default: 10), the loop transitions to monitor-only mode and alerts the operator. This gate was validated empirically in the Edge deployment (Section 11.5): at iterations 112-127, all remaining work required changes in an external dependency (SDK/Python) unreachable from the Edge (TypeScript) codebase. The circuit breaker fired 11 times, and the loop correctly recommended stopping. When the dependency blockers were resolved at iteration 128, the loop immediately resumed productive work (14 PRs in one batch). The starvation gate validates the spec-anchored design: when the specification surface is fully covered, the loop stops rather than inventing work. Three control counters — starvation, drain entries, and no-work loops — are persisted to disk, so the loop retains its operational position across process restarts.

Of the five gates, drain mode and starvation are fully automatic. Regression failure halts only after consecutive iteration failures exceed a configurable threshold (default: 3). Drift and canary escape currently warn rather than pause — making them advisory gates that rely on operator attention. These gates ensure the loop cannot degrade the product faster than it improves it. The loop is allowed to run autonomously *because* the gates provide a safety net.

8 System Architecture

8.1 The Six-Phase Loop

The Kitchen Loop is orchestrated by a shell framework that manages state transitions, git worktree isolation, and error recovery. The core logic is delegated to specialized AI skills, each encoding a repeatable, autonomous workflow.



Backlog (~15 min): Evaluates urgency and coverage gaps, promotes candidates to the work queue, generates new scenario tickets when supply runs low.

Ideate (~15-45 min): Selects a scenario, implements it as a real user would, runs it against the test environment, documents what breaks. Optionally passes through an external feasibility check.

Triage (~5-10 min): Converts findings into labeled, prioritized tickets with root cause hypotheses and file pointers. Deduplicates against existing tickets, and reopens tickets whose prior fix PRs were closed without merging.

Execute (~30-60 min): For each top-N urgent ticket: creates a feature branch in an isolated worktree, implements the fix, writes tests, opens a PR. Includes backpressure control.

Polish (~10-90 min): PR hardening and merging through a graduated state machine. Each PR is tracked with an attempt counter; after a configurable number of failures (default: 1), it is labeled `needs-attention` and excluded from future processing. Operators can raise the threshold to enable multi-attempt escalation with follow-up ticket creation and failure classification. PRs blocked by security or architectural concerns are retired — closed with a comment routing the ticket back to the backlog. Before any merge, a TOCTOU-safe deletion check verifies no files were unexpectedly removed.

Regress (~40-150 min): Runs the regression oracle (read-only — no fixes). Updates loop state directly to the base branch. Promotes confirmed patterns to durable memory. Produces iteration summary with drift metrics. Before updating shared state files (e.g., `loop-state.md`), the Regress phase re-reads the current version to prevent stale overwrites from interrupted or concurrent runs.

8.2 Skills as Prompts-as-Code

Each phase is implemented as a **skill** — a structured markdown file encoding a repeatable workflow in natural language. Skills are version-controlled, portable across LLM providers, and improved in natural language without deployment. A Kitchen Loop-compatible system needs standardized skill interfaces:

Skill	Input	Output	Must Guarantee
backlog	Ticket state	Promoted + new scenario tickets	No duplicate tickets created
ideate	Scenario criteria	Experience report + scenario	Runs against real test env
triage	Experience report	Prioritized tickets w/ file pointers	Deduplicates against existing
execute	Ranked ticket list	Feature branch + PR per ticket	Every PR includes tests
regress	Full codebase state	Pass/fail report + drift metrics	Runs regression oracle

Teams adopting the Kitchen Loop implement these interfaces for their domain. The orchestration — phase sequencing, timeout management, error recovery — is domain-independent.

8.3 Execution Modes

Mode	Purpose	When to Use
<i>Framework modes (domain-independent):</i>		
strategy (default)	Full cycle: ideate + execute + regress	Standard operation
user-only	Rapid ideation loops to fill the backlog	Backlog empty, need discovery
dev-only	Implementation-focused loops to drain the backlog	Backlog full, need throughput
drain (auto)	Polish-only with increased PR throughput	Auto-triggered by backpressure
regress-quick	Shortened regression: one scenario per platform	Every iteration (fast feedback)
ui	Browser-driven user flows with visual checkpoint verification	Web application testing
<i>Domain-specific modes (DeFi strategy framework example):</i>		
backtest	Backtesting pipeline instead of on-chain execution	Backtesting stress-testing
exploration	Explore new protocol/chain coverage gaps	Coverage discovery

8.4 The Shared Memory: Ticket Management as PM Layer

The central nervous system is the project management tool — used not as a lightweight task tracker but as the shared, durable memory layer between the AI loop and human contributors.

Both AI and human operators write to the same backlog. The Execute phase cannot tell the difference — it queries “top N urgent unblocked tickets” and works through them. This creates **parity between AI and human judgment**: the backlog is the truth, neither source is privileged.

Field	Convention	Why
Labels	bug, feature, improvement, exploration	Picks bugs first, then balances
Priority	critical, high, medium, low	Respects human priority, no override
State	Backlog → Todo → In Progress → In Review → Done	PRs auto-link to tickets via title
Dependencies	blocks/blockedBy	Skips blocked tickets automatically

8.5 The Bar: Production-Readiness Standard

The Kitchen Loop applies “**The Bar**” — a project-specific quality standard defined in a customizable `quality.bar_file`. The framework ships a generic template covering code quality, PR standards, safety, and documentation; each deployment customizes for its domain. This approach is supported by Abtahi and Azim (2025), who find that categorizing quality issues by type achieves 100% resolution vs. 71.6% for bulk processing [5] — the Kitchen Loop’s phase-based architecture naturally decomposes enforcement into categories (functional correctness in Execute, style in Polish, regression safety in Regress).

For example, the DeFi strategy framework (Case Study A) instantiates The Bar as five domain-specific principles:

1. **Production Ready** — No TODOs, shortcuts, or “we’ll fix later.” Ship-quality on merge.
2. **Zero Hardcoding** — All configuration from resolvers and registries, never literals.
3. **Million-Dollar Safe** — Every external call handled, amounts rounded safely, no silent fallbacks.
4. **Hedge-Fund Serious** — Precise arithmetic everywhere, meaningful test assertions, conservation checks in all failure paths.
5. **UX First, Safety Always** — Clean, declarative APIs; non-bypassable safety gates; secrets redacted from logs.

These principles are enforced through multi-model review tribunals — every PR is evaluated against them before merge. Other domains would define their own principles (e.g., HIPAA compliance for healthcare, WCAG conformance for web accessibility).

9 The Discussion Manager: Structured Multi-AI Deliberation

9.1 Beyond Review Tribunals

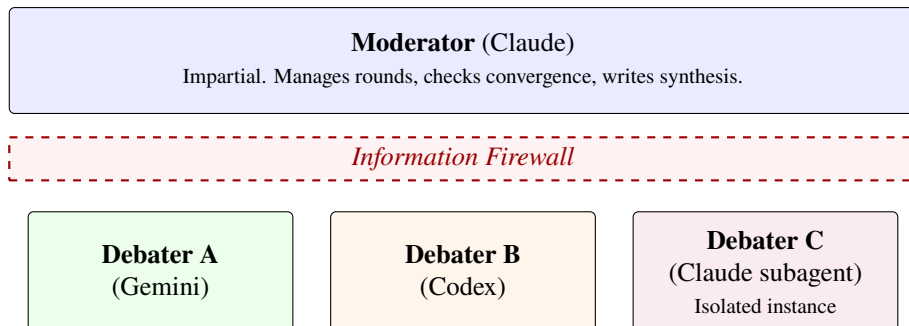
Section 4.6 introduced multi-model review tribunals — three independent AI reviewers evaluating the same artifact in parallel. The Discussion Manager extends this concept into a full **structured deliberation system**: a multi-round, multi-model debate framework where heterogeneous AI agents argue substantive positions, challenge each other’s reasoning, and converge toward actionable decisions through an impartial moderation layer.

Where review tribunals answer “is this code correct?”, the Discussion Manager answers harder questions: “should we build this at all?”, “which of three competing architectures best serves the specification surface?”, and “what are we not seeing?” These are the judgment-intensive decisions that the Kitchen Loop encounters in its Ideate and Triage phases — decisions where a single model’s blind spots can compound into wasted iterations.

The Discussion Manager operationalizes what Cai et al. (2025) identify as the “Debate-Based Cooperation” pattern — adversarial argumentation to surface errors — which appears in only 4 of 94 surveyed multi-agent systems (4.3% adoption) [8]. The low adoption rate, despite theoretical value for error discovery, suggests this is an underexplored design space. The Kitchen Loop’s production implementation, with its anti-sycophancy safeguards and empirical validation across 23 discussions, represents one of the first operational deployments of this pattern at scale.

9.2 Architecture

The Discussion Manager uses a **centralized moderator architecture** with an information firewall between moderator and debaters:



Key design decisions:

- **Heterogeneous models:** Three different model families (Gemini, Codex/GPT, Claude) ensure genuine perspective diversity. Each model brings different training data, different reasoning patterns, and different failure modes. Homogeneous debate (three instances of the same model) rapidly converges to groupthink.
- **Centralized moderation:** The moderator orchestrates turn order, checks convergence, and writes the synthesis — but never injects opinions into the debate itself. Centralized judge structures outperform peer-review structures for debate quality (Yao et al., 2025).
- **Claude subagent isolation:** When the moderator is Claude, the Claude debater runs as an isolated subagent with no access to the moderator’s reasoning, notes, or context. The subagent sees only the debate prompt. This is the information firewall that prevents the moderator’s framing from contaminating the debate.
- **Code grounding:** For technical discussions, debaters receive codebase context (specific files, architecture documents, or topic-aware summaries). Gemini gets autonomous file-search access; other debaters get relevant content injected into their prompts. This anchors debate in verifiable facts rather than rhetorical polish.

9.3 Empirical Evidence: 23 Production Discussions

The Discussion Manager’s design was shaped by Yao et al.’s (2025) research on **sycophancy in multi-agent debate** [14], which identifies three failure modes: *sycophancy*, *disagreement collapse*, and *negative agreement*. Key design-relevant findings: heterogeneous models outperform homogeneous; centralized judges outperform peer-review; and capping debate at 2-3 rounds preserves productive disagreement.

We evaluated the system against these criteria using a corpus of **23 production discussions** conducted across three codebases over a 4-week period. Two independent meta-analyses — one by Claude (moderator) and one by Codex (independent evaluator) — mitigate self-assessment bias.

Evaluation against paper metrics:

Dimension	Value
Total discussions	23 unique discussions across 3 codebases
Discussion types	Architecture/PRDs (10), Code reviews (7), Strategy (5), Exploration (1)
Typical participants	3 models (Gemini, Codex, Claude); 1 discussion used 2 models
Typical rounds	3 (range: 1–10)
Convergence	2/23 formally converged; 21/23 hit max rounds
Conclusions	100% “Agreed” — zero “Agreed to Disagree”

Criterion	Measured	Paper’s Ideal	Assessment
Disagree. Collapse Rate (DCR)	~0% (all converge)	Low but non-zero	Red flag — universal convergence implausible
Sycophancy Score (SS)	35–50	<20	Moderate-high — mitigated by code grounding
Negative Agreement Rate (NAR)	~25%	<10%	Moderate — Claude concedes readily
Evidence Quality	85/100	High	Strong — code grounding is best feature
Perspective Diversity	60/100	High heterogeneity	Moderate — good diversity, rigid roles
Round Efficiency	~50% productive	>80%	Low — 2–3 wasted rounds per discussion

Findings from both meta-analyses:

Strengths. Code grounding anchors every discussion in specific line numbers and verifiable API behavior, strongly mitigating the “rhetorical polish without substance” failure mode. Actionability is consistently high (phased remediation plans, PRDs, file-level recommendations), and raw transcripts confirm substantive disagreement, corrections, and explicit position changes.

Weaknesses. Disagreements resolve by one party conceding rather than genuine synthesis (premature convergence). The first speaker sets the problem’s ontology, with later speakers refining rather than challenging it (sequential anchoring). Convergence tracking is cosmetic — counting self-reported DISAGREEMENTS items rather than verifying semantic resolution. No discussion concluded with “do not build this,” revealing a structural bias toward action.

Meta-analyst disagreement. The independent Codex meta-analysis challenged Claude’s “100% ratification” claim (one discussion lacked ratifications in the JSON) and “monotonic convergence” pattern (some discussions showed non-monotonic trajectories).

Mitigations implemented. Based on these findings and Yao et al.’s recommendations, the Discussion Manager now implements: blind opening rounds (eliminating first-speaker anchoring), a structured issue register (replacing cosmetic convergence counting), explicit proposal ratification, a kill gate requiring a “do not build this” argument before proceeding, and full transcript preservation for reproducible meta-analysis.

9.4 Implementation and Open Challenges

The Discussion Manager is implemented as a Python orchestrator (`discuss.py`, open-sourced in the companion repository) that manages conversation creation, turn execution, convergence checking, ratification, and report generation. It is designed for use at three integration points: (1) **Ideate phase** for architectural decisions, (2) **Code review** in audit mode, and (3) **Retrospective** comparing implementation against agreed designs. In the current open-source release, the Discussion Manager is invoked manually; automated orchestrator integration is planned.

Despite the epistemic improvements, several challenges remain:

- **Cost:** Three models for 3–5 rounds at ~400 words per turn consumes 15,000–25,000 tokens per model per discussion. With the ~50% round efficiency observed in the corpus, approximately 40% of token spend is on convergence dynamics rather than novel insight. The cost implications for scaling are discussed in Section 11.4.

- **Role calcification:** Even with heterogeneous models, each model tends toward a fixed role (Gemini as breadth analyst, Codex as bug hunter, Claude as compromise broker). Random role assignment per discussion can help, but the underlying model tendencies persist.
- **Evaluation circularity:** When the moderator is Claude and one meta-analyst is also Claude, self-assessment bias is difficult to fully eliminate. The independent Codex meta-analysis mitigates this but does not resolve it entirely.
- **Scaling beyond three:** The current protocol is optimized for three debaters. Scaling to more participants increases round duration and convergence complexity without proportional improvement in perspective diversity.

These are active research questions. The framework’s modular design — separating orchestration, moderation, and debater execution — allows incremental improvement without architectural overhaul.

10 Case Study A: DeFi Strategy Framework

10.1 The Product and Its Specification Surface

The first validation target is a production DeFi strategy framework built by Almanak (<https://almanak.co>), the Almanak SDK for DeFi Quants — a public, open-source repository.² A quant writes a strategy class, declares high-level intents (swap, LP open/close, borrow, supply), and the framework compiles those intents to on-chain transactions, executes them, parses receipts, and updates state.

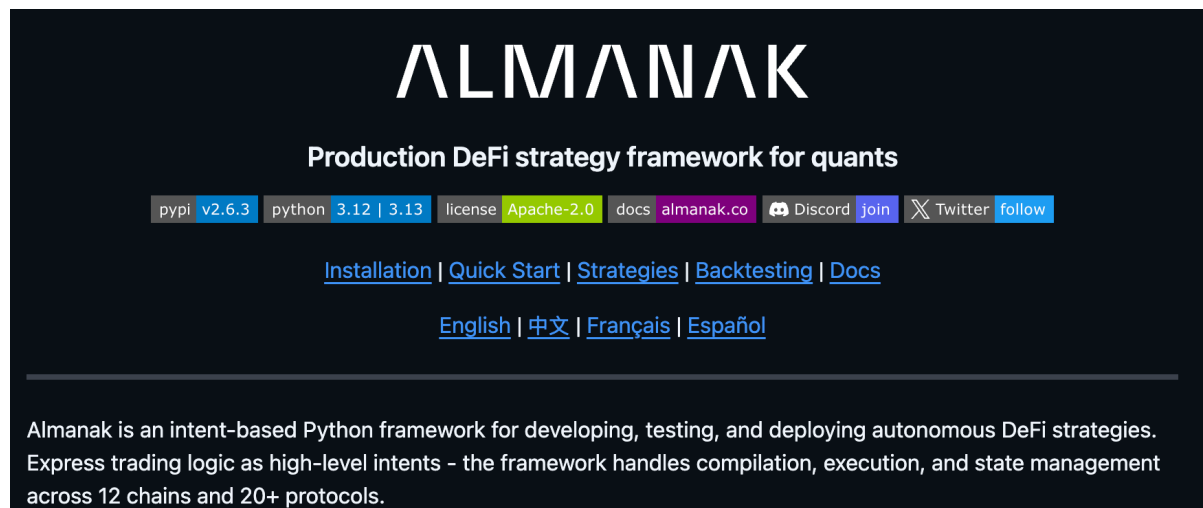


Figure 3: The Almanak SDK repository: a production DeFi strategy framework supporting 14 chains and 30+ protocol connectors.

The specification surface:

Dimension	Count	
Supported chains	14	(13 EVM + Solana)
Protocol connectors	30+	(13 core + Aave, Compound, GMX, Morpho, Lido, ...)
Intent types	21	(Swap, LP Open/Close, Borrow, Supply, Perp, Stake, ...)
Cross-combinations	~1,000	(chain × protocol × intent)

10.2 The Regression Oracle: Fork Execution

The regression oracle is a suite of demo strategies executed on **Anvil** (a local EVM chain fork). Each strategy runs against a fork of the target chain, compiles intents to transactions, executes them, and verifies on-chain state changes.

A PASS requires at least one on-chain transaction with verified balance deltas. A PASS (HOLD) is valid (market conditions didn't trigger a trade). A FAIL is always a regression until proven environmental.

The 4-layer verification pattern applied:

Layer	Implementation
Compilation	<code>compiler.compile(intent) → assert status SUCCESS</code>
Execution	<code>orchestrator.execute(bundle) → assert success on-chain</code>
Receipt Parsing	Protocol-specific parser extracts amounts, position IDs

²<https://github.com/almanak-co/sdk>

Layer	Implementation
State Deltas	<code>get_token_balance()</code> BEFORE and AFTER, assert exact expected changes

10.3 Results

Metric	Value
Loop iterations completed	122+
Merged pull requests	728+
Unique tickets resolved	350+
Lines of code added	~250,000
Unit tests (start)	~6,400
Unit tests (current)	10,913
Demo strategies	62 (up from 13)
Incubating strategies	183
Chains covered	13 EVM + Solana
Protocols exercised	30+ (Uniswap V3/V4, Aave, Morpho, Curve, Pendle, ...)
Intent types exercised	All 21 (SWAP, LP_OPEN, LP_CLOSE, BORROW, SUPPLY, ...)
Consecutive zero-bug iters	16 (iterations 55–70)
Regressions (loop-merged)	0

These results span 122+ iterations over 5 weeks (Feb 18 – Mar 23, 2026).

10.4 Phase Progression

Phase 1: Bug Discovery (Iterations 1–23). Dominated by “obviously missing” bugs — table-stakes features every user would hit immediately:

Representative Finding	Impact	Iter
API method documented but not exposed to users	<code>AttributeError</code> on first call	12
Gas price cap correct for one chain, blocks all txs	All strategies on one chain fail silently	18
Protocol router config missing for entire protocol	Blocks swap compilation everywhere	50
Native token symbol missing for two chains	Blocks ALL strategies on those chains	51

Phase 2: Coverage Expansion (Iterations 24–54). Focus shifted to new protocol connectors, new chains, and multi-protocol composability. Notable: first lending protocol on a new chain (37 unit tests); discovery of router interface V1 vs V2 mismatch causing silent reverts; first multi-protocol strategy (11 transactions across 4 intent types, zero bugs); first indicator-driven LP strategy validating the market data → position sizing pipeline.

Phase 3: Maturity (Iterations 55–71). The system reached maturity: **16 consecutive zero-bug ideation phases**. Bug discovery shifted from “obviously missing” to configuration gaps and infrastructure reliability issues. New capabilities shipped autonomously: a direct-action CLI, support for a new blockchain, advanced demo strategies.

Phase 4: Deep Verification (Iterations 72–89). The loop transitioned from coverage breadth to verification depth, finding production-severity bugs that traditional testing would miss:

Finding	Severity	Impact	Iter
Pendle PT sell uses 1:1 exchange rate, but PT trades at ~19% discount	HIGH	Every PT sell at normal slippage (0.5-10%) would revert with <code>INSUFFICIENT_TOKEN_OUT</code>	89
BTC.b completely missing from Avalanche token resolver	HIGH	Any BTC.b strategy on Avalanche fails at compilation	83
Uniswap V4 adapter silently falls back to 18 decimals	MEDIUM	Incorrect balance calculations for non-18-decimal tokens	80-81

A single 7-iteration overnight batch (iterations 83-89) produced **18 PRs, 370+ new tests, and 5 bug discoveries** — demonstrating sustained throughput even at high iteration counts. Independent multi-model code review (Claude pr-auditor + Codex/GPT-5.4) rated the merged code as **STRONG** across all quality principles.

Regression stability under load: Despite 20+ PRs merged between checkpoints, the demo strategy suite maintained 100% pass rate across all recent iterations. Quick regression tests 4 chains in under 2 minutes.

Phase 5: Operational Maturity (Iterations 90–122). The loop entered a sustained production rhythm with improved strategy diversity, predictable throughput, and a stabilizing PR backpressure cycle:

- **Cross-chain validation at scale:** First BSC swap (iter 92), first Sonic execution (iter 100), first Polygon multi-protocol composition (iter 94). 13 EVM chains + Solana now covered.
- **Safety-critical discoveries:** Fabricated Uniswap V4 addresses (P0, iter 93), wrong Ethena unstake selector (iter 99), CryptoSwap zero-slippage protection (iter 95), price impact guard for zero-liquidity pools (iter 122). These are deep bugs that traditional testing misses.
- **Infrastructure healing cycle matures:** The loop diagnosed and fixed its own PR merger timeout (iter 101), stale loop-state detection (iter 107+), and demo regression blind spots (iter 74). Each infrastructure crisis resolved faster than the previous one.
- **Zero-bug iterations become common:** Iterations 94, 108, 115, 116, and 121 all passed with zero new bugs, validating connector portability across chains.

A single 3-iteration overnight batch (iterations 120-122) produced 9 PRs, 12 merges, ~249 tests, and a systemic price impact guard — demonstrating sustained throughput at high iteration counts.

10.5 The Self-Healing Property

The loop discovered and fixed its own infrastructure bugs through its standard process — no human diagnosis required:

- **Apple Silicon memory bug:** PR merge automation used the wrong memory page size (4096 instead of 16384), causing available memory to read 4x too low. This blocked merging for 5 consecutive iterations. The loop-review skill identified the pattern; Execute fixed it; Regress confirmed the fix.
- **Codex feasibility checker:** The external AI cooperation model (Section 12.3) matured through the same loop cycle — strict response contracts, timeout handling, and salvage paths were all refined by the loop itself.
- **Ideate persistence bug** (iter 13-19): Six iterations of lost deliverables (reports not committed to git) before the loop identified the pattern, filed a ticket, and shipped a fix that decoupled the commit/push lifecycle. Verified stable from iter 19 onward.

- **PR Manager death spiral** (iter 71-73): Permanent `needs-attention` labels caused a 62-hour deadlock with zero merges. The loop review diagnosed the root cause; subsequent iterations added TTL-based label recovery and a circuit breaker. Full recovery by iter 74 (5/5 PRs merged).
- **Demo regression blind spot** (iter 53-74): Twenty iterations of blind demo testing because `ALCHEMY_API_KEY` was missing in worktree environments. The loop detected the pattern, shipped a `.env` copy to worktrees with a pre-flight key check, and confirmed the fix in iter 75.

This property — the loop improving its own tooling through the same process it uses to improve the product — is further demonstrated in Case Study B (Section 9) and elaborated in Section 13.

11 Case Study B: Signal Intelligence Platform

11.1 The Product and Its Specification Surface

The second validation target is Edge, a DeFi signal intelligence platform built by Almanak (<https://almanak.co>), available at <https://app.almanak.co>. Edge comprises 46+ detection agents that scan on-chain and off-chain data sources, produce structured signals, and feed a synthesis pipeline that correlates signals into actionable intelligence.

The specification surface:

Dimension	Count
Detection agents	46+
Signal types	77 (all verified)
Verifier families	22
Cross-combinations	~1,078 (agent × signal type)

11.2 The Regression Oracle: Quality Gates + Canaries

Signals are ephemeral and non-deterministic — they cannot be “replayed” like strategy executions. The regression oracle combines deterministic quality gates with anti-signal canaries:

4-layer quality gate:

Layer	Name	Method	What It Catches
L1	Structural	Schema validation	Format errors, invalid enums, missing metadata
L2	Factual	API cross-reference (DefiLlama, Snapshot, CoinGecko, GeckoTerminal, on-chain RPCs)	False claims, wrong numbers
L3	Temporal	Dedup + timestamp analysis	Stale signals, zombie duplicates
L4	Cognitive	LLM tribunal (3 models)	Bad reasoning, wrong conclusions from correct data

4-tier anti-signal canaries (24 total + 3 API degradation):

Tier	Canaries	Catch Rate (iter 1)	Catch Rate (iter 163)
Tier 1 (Obviously Bad)	6	100%	100% (zero escapes across 163 iters)
Tier 2 (Shadow)	5	33%	100% (fixed iter 124)
Tier 3 (Adversarial)	5	67%	100%
Tier 4 (Mixed True/False)	5	N/A (added ~iter 100)	100% (fixed iter 124)
API Degradation	3	N/A (added later)	100% (3/3)

11.3 Results

Metric	Value
Loop iterations completed	163

Metric	Value
Tickets created	200+
Pull requests merged	366
Signal type coverage	77/77 (100%) — up from 33/36 at iter 1
Verifier coverage	100% (22 verifier families) — every signal type has a dedicated L2 factual verifier
Non-functional agents identified	18 (managed via exclusion list)
L1/L2/L3 pass rates	100% / 100% / 100% (up from 85% / 76% / 85% at iter 1)
L2 unverifiable rate	0% (down from 9-31% at iter 1)
Canary health	100% all 4 tiers (zero Tier 1 escapes across 163 iterations)
Test cases	2,171 across 152 test files
Average iteration speed	~45 min avg (25-35 min signal-focused, 80-230 min full, 15-20 min circuit-breaker skip)
PR merge rate	97% (366/377)
Median time to merge	1.1 hours
Regressions from loop-merged code	0

These results span 163 iterations over 2.5 weeks.

11.4 Quality Gate Maturity Trajectory

The most compelling evidence of the loop’s self-improving quality infrastructure:

Metric	Iter 1-10	Iter 50-60	Iter 128-135	Iter 163
L1 pass rate	85-100%	100%	100%	100%
L2 pass rate	76-91%	90-97%	100%	100%
L2 unverifiable	9-31%	3-10%	0%	0%
L3 pass rate	85-100%	92-100%	100%	100%
Verifier coverage	~92% (33/36)	~100%	100% (72/72)	100% (77/77)
PR merge rate	~0%	~85%	100%	97%
Open PR backpressure	0-3	3-10	0	0-3

Key insight: In this deployment, the quality gates improved monotonically. No regression in pass rates was detected by the oracle after any of the 366 merged PRs across 163 iterations. This suggests that autonomous code changes can maintain (and improve) quality invariants without continuous human oversight, given sufficient verification infrastructure.

11.5 Phase Progression

The Edge deployment traversed four distinct phases, mirroring the SDK’s progression but at higher iteration speed:

Phase 1: Bootstrap (Iterations 1-12, Days 1-2)

Detection engine without self-correction capability. 12 PRs created, 0 merged — the loop could find bugs but couldn’t fix them. PR manager and polish phase activated; first merges in iter 11. Quality trajectory: L1 85→100%, L2 69→96%, L3 85→100%.

Phase 2: Steady-State Improvement (Iterations 13-87, Days 2-6)

Productive self-improvement with monotonically improving quality. 200+ PRs merged. The loop

identified and fixed 7 infrastructure bugs through its own Ideate→Triage→Execute cycle: chain-Valid permanently fixed, drift persistence migrated to file-backed storage, verifier families expanded, canary Tier 2 fixed, circuit breaker added, non-functional agent exclusion list created. L1/L2/L3 all reached and held 100%.

Phase 3: Starvation (Iterations 88-127, Days 6-9)

All remaining tickets required SDK (Python) changes unreachable from the Edge (TypeScript) code-base. The circuit breaker fired on 10+ consecutive iterations. Two of three reviewers recommended STOP THE LOOP. This was correct behavior: the loop did not degrade or invent work — it recognized exhaustion and recommended stopping. Quality gates maintained 100% throughout.

Phase 4: Recovery and Expansion (Iterations 128-163, Days 9-17)

New work became available after SDK blockers resolved. 100+ PRs merged. 20+ new agents tested, LPOpportunityAgent fixed, ArbitrageAgent activated, TechnicalAgent added. Canaries reached 100% across all 4 tiers by iteration 124 and held for 40+ subsequent iterations.

11.6 Key Findings (Early Iterations)

- A critical cooldown bypass needed — 33+ agents unreachable due to dedup, blocking 75% of fleet from quality testing
- A production agent using `Math.random()` simulation instead of real on-chain data — causing temporal failures with stale signals from tokens launched over a year ago
- A “dead” agent registered in config but not in the agent system — producing zero signals silently
- The loop’s own quality gate conflating “unverifiable” with “failed” — a false positive the loop discovered and fixed through its own triage process

11.7 The Self-Correction Flywheel (Iterations 128-135)

At maturity, the loop’s most powerful property is **iterative self-correction** — it doesn’t just find bugs, it fixes them, verifies the fixes, and catches when fixes are incomplete.

The OnChainSentinel Fix Chain:

Iter	What Happened
130	Ideate discovers OnChainSentinel hangs the pipeline via WebSocket constructor blocking. Creates ticket.
130	Execute ships PR: 5-second timeout guard around <code>getNetwork()</code> + adds agent to exclusion list.
131	Ideate retests. Discovers the fix was incomplete — the <code>WebSocketProvider</code> constructor itself blocks before <code>getNetwork()</code> is ever called. Reopens ticket.
131	Execute ships PR: wraps entire WS provider init in a single timeout with provider cleanup on expiry. Adds destroy-on-timeout test.
131	Regress confirms: build passes, 27 signals produced, L1/L2/L3 100%, no regressions.

This is a 3-iteration cycle from discovery → incomplete fix → complete fix → regression verification, with zero human intervention. The loop caught its own incomplete fix and iterated until the root cause was fully addressed.

The ArbitrageAgent Data Pipeline Resurrection:

The ArbitrageAgent went through a 3-PR fix chain across multiple iterations:

1. **PR #278** — Replaced dead The Graph hosted subgraph URLs with GeckoTerminal API (The Graph deprecated its free hosted service — a real external platform change the loop detected)
2. **PR #282** — Fixed incorrect `base_token_price_usd` vs `quote_token_price_usd` field usage that produced nonsensical 3,499× spreads

3. **PR #286** — Lowered `ARBITRAGE_PROFIT_MIN_PERCENT` from 1% to 0.1% after observing 2 consecutive zero-signal runs (1% required a 1.6% gross spread on Ethereum — far above real market conditions of 0.1-0.5%)

Each iteration peeled back one more layer of the problem. The loop doesn’t give up after the first fix — it keeps testing until the agent actually produces signals.

11.8 Triage Intelligence: The Classification System

The triage phase demonstrates sophisticated classification that goes beyond binary “working/broken”:

Classification	Example	Response
Non-functional (confirmed)	RWARiskAgent: hardcoded stub data (backingRatio=1.0)	Add to exclusion list immediately
Non-functional (suspected)	LPRotationAgent: 0 signals, 1st observation	Monitor — needs 2+ confirmations before excluding
Cold-start	DevActivityAgent: needs warm baseline from prior runs	Don’t exclude — works in scheduled mode
Timing-dependent	BribeTrackingAgent: no active vote rounds	Don’t exclude — market conditions, not a bug
Deprecated	YieldAgent: converted to return-empty stub	Add to exclusion list

Self-correcting classification: In iteration 131, triage classified `DevActivityAgent` as “non-functional.” In iteration 132, it corrected this diagnosis — the agent has a working GitHub API integration and is actually cold-start, not broken. The ticket was updated, and the agent was NOT added to the exclusion list. Self-correction extends beyond code fixes to the classification system itself.

11.9 Infrastructure Self-Healing (Iterations 128-135)

The loop identified and fixed 6 problems in its own infrastructure during a single 8-iteration batch:

Finding	Fix	Impact
<code>RESULTS_DIR</code> undefined in <code>pr-manager.sh</code>	Grep prep log instead	Gate rejection memory completely non-functional
<code>gh --jq --arg</code> invalid syntax	Pipe to <code>jq --arg</code>	Shell command silently failing
Missing iter 127 row in <code>loop-state.md</code>	Backfill + verification step	History gap invisible to drift detection
<code>synthesisPipelineDead</code> false positive	<code>localDev</code> flag for drift suppression	9 iterations of wasted CRITICAL escalation
Deployment drift (local main stale)	Pre-flight detection	All merged fixes untested for 5+ iterations
<code>NON_FUNCTIONAL_AGENTS</code> set out of sync	Sync with confirmed findings + expose via API	Ideate kept selecting dead agents

As in Case Study A (Section 8.4), the loop improves at two levels: the product and its own infrastructure.

11.10 Speed Difference: Two Domains, Same Loop

Phase	Strategy Framework	Signal Platform	Speedup
Ideate	10-15 min	1-2 min	~8x
Triage	5-8 min	<1 min	~8x
Execute	25-60 min	10-30 min	~2x
Regress	40-150 min	2-3 min	~30x
Total	80-230 min	~15-35 min*	~5-15x

*Signal-only iterations (ideate + triage) complete in ~5 min; full 6-phase iterations take 80-230 min. The ~15-35 min figure reflects signal-focused iterations including execute and polish. Circuit-breaker skip iterations complete in 15-20 min; the average across all 163 iterations was ~45 min.

The signal platform iterates faster because signal quality verification is API-bound, not execution-bound. This speed advantage compounds over time: at 163 iterations, the signal platform has performed more quality gate evaluations than most human QA teams accomplish in a year.

12 Cost of Operation

12.1 Tooling Spend

A common concern: “How much does it cost to run an AI agent continuously?” The answer is surprisingly modest: the Kitchen Loop runs entirely on **flat-rate subscriptions**, not metered API calls.

Cost Component	Monthly Cost	Notes
Claude Code Max (20x plan)	\$200	Primary agent for all 6 phases — unlimited usage within plan
Codex (subscription)	\$20	External feasibility checker + tribunal reviewer
Gemini (subscription)	\$20	Tribunal reviewer for multi-model audits
CodeRabbit	~\$15	Automated PR code review
Anvil / Foundry	\$0	Open-source, runs locally
External data APIs	~\$0-50	CoinGecko, DefiLlama (free tiers sufficient)
CI compute	~\$50-100	GitHub Actions minutes
Total	~\$305-405/month	

This is the total cost for running the Kitchen Loop across **both** production systems simultaneously — 285+ iterations, 1,094+ merged PRs, 700+ tickets resolved. A senior engineer costs \$12,000-25,000/month fully loaded; the Kitchen Loop’s monthly tooling cost is **~2% of a single engineer’s cost**. Critically, this includes the quality infrastructure (unbeatable tests, UAT gates, regression oracles) that prevents the complexity-driven velocity erosion documented by He et al. [11] and Becker et al. [12].

Metric	Kitchen Loop	Single Engineer	Ratio
Monthly cost	~\$350	~\$15,000	~43x cheaper
PRs merged/month	600+	~15-25	~30x more
Coverage scenarios tested/month	150+	~10-20	~10x more
Cost per merged PR	~\$0.38	\$600-1,000	~1,800x cheaper

Because cost is fixed regardless of iteration count, the marginal cost of additional coverage is effectively zero.

12.2 Test Suite Growth and Runtime

As the test suite grows, regression time grows. This is a real scaling concern:

Metric	Start (iter 1)	Current (iter 122)	Growth Rate
Unit tests	~6,400	10,913	+70% over 122 iterations
Full regression time	~90 min	~150 min	~0.5 min/iteration growth
Quick regression time	~25 min	~40 min	~0.12 min/iteration growth
Demo strategies	13	62	~0.40/iteration
Incubating strategies	0	183	~1.5/iteration

Mitigation strategies already in use: - `--regress-quick` mode: one scenario per platform (~40 min vs ~150 min) - Full regression runs weekly, quick runs every iteration - Parallel test execution for unit tests (pytest-xdist) - Test pruning: deprecated/redundant strategies archived, not accumulated

Projected ceiling: At current growth rates, full regression would reach ~4 hours by iteration 200. This is manageable with parallelization (dynamic port allocation for multiple test environments), sharding (split the coverage matrix across N parallel loops), or sampling (statistical coverage sampling rather than exhaustive every iteration).

12.3 Business Outcomes

For readers focused on outcomes: 50+ features shipped, 200+ bugs found before users (including fund-safety issues), coverage matrix fill rate improved from ~5% to ~50% of 1,800 combinations, and test suite grew +70%. Detailed breakdowns are in Sections 8.3 and 9.3.

13 Production Safety Record

13.1 Incidents and Ill Effects

A critical question for any autonomous system: **has it caused harm?**

Concern	Record	Details
Production downtime	0 incidents	Loop operates on isolated branches + test environments (Anvil forks). No direct production access.
Security incidents	0 incidents	Loop has no access to production keys, wallets, or user data. Test wallets use Anvil defaults.
Data loss	0 incidents	Git worktree isolation prevents branch contamination. All changes are reversible PRs.
Regressions shipped to main	0	Quality gates (multi-model review + regression oracle) catch issues before merge.
Cost overruns	0	Monthly tooling cost is bounded and predictable (~\$305-405/month across both systems, ~\$1.50/iteration).

13.2 Why Zero Incidents?

The clean safety record follows from **architectural isolation**: no production access (test environments only), branch isolation (git worktrees on feature branches), automated pause gates (Section 5.5), and backpressure control. These are structural properties, not luck.

13.3 Known Limitations (Honest Assessment)

Limitation	Impact	Mitigation
Anvil-only testing	Loop has not been exercised with real mainnet transactions	Mainnet mode exists but requires explicit operator approval + real gas
API key dependency	Some scenarios require external API keys not in standard config	Produces PASS (caveat) instead of clean PASS — a known quality ceiling
Single-threaded execution	Gateway port constraint limits to sequential strategy runs	Future: dynamic port allocation for parallel execution
PR merge automation fragility	The Polish phase has been the most failure-prone component	Each failure mode has been fixed through the self-improvement cycle
Cooldown phantom failures (signal platform)	Agents with cooldown periods inflate dead-agent counts	Distinguishing “cooldown active” from “agent broken” is tracked as an open ticket

This section is intentionally honest. The loop is not infallible. But its failure modes are *operational* (merge automation bugs, API key management) rather than *safety-critical* (data loss, fund loss, production outages). The architectural isolation ensures that operational failures waste iteration time, not user trust.

13.4 Human-in-the-Loop Cost Constraints

The loop is autonomous end-to-end: it generates scenarios from the specification surface, fills its own backlog, triages findings into tickets, implements fixes, and auto-merges PRs after multi-model review and CI. No human is in the critical path during normal operation. The human role reduces to (1) initial specification surface definition, (2) occasional strategic steering, and (3) **supervisory intervention** when the loop encounters failure modes it cannot self-correct.

In practice, the most common intervention trigger was *merge conflict resolution gone wrong*: LLMs resolving git conflicts would sometimes silently undo work from previous commits or PRs, requiring a human to notice the regression and revert. GitHub infrastructure issues (API rate limits, webhook failures, stale branch state) were the second most frequent cause. Section 12.2 frames the steady-state human cost as ~30-60 minutes per week once calibrated — but this assumes the loop is running smoothly. Intervention spikes during infrastructure instability.

The practical scaling constraint is *environmental*: test environment throughput (Anvil fork startup time, API rate limits), CI pipeline capacity, and the cost of multi-model review tokens per PR. Drain mode (Section 5.5) throttles output when PR backpressure exceeds a threshold, ensuring the loop does not outrun its own merge infrastructure.

The Discussion Manager adds a separate cost dimension. Three models debating for 3-5 rounds at ~400 words per turn consumes 15,000-25,000 tokens per model per discussion. With the ~50% round efficiency observed in our 23-discussion corpus (Section 7.3), approximately 40% of deliberation token spend produces convergence dynamics rather than novel insight. At current subscription pricing this cost is absorbed into flat-rate plans, but metered API usage would make frequent deliberation expensive. Per-iteration compute cost breakdown beyond the aggregate \$0.38/PR figure has not yet been instrumented — this is a gap we intend to address in future work.

These constraints do not invalidate the framework, but they bound its applicability: the Kitchen Loop is most efficient in domains where (a) the specification surface is well-defined upfront, (b) the regression

oracle provides high-confidence automated verification, and (c) the human’s role can be genuinely asynchronous rather than synchronous with each iteration.

13.5 Open Problems

Our deployments expose four open problems that we believe warrant dedicated research:

- **OP1: Oracle Transfer.** The Kitchen Loop relies on a bespoke regression oracle per domain (chain fork execution for DeFi, quality-gate pipeline for signals). Automatic generation of regression oracles from natural-language specifications — eliminating the per-domain engineering cost — is an unsolved challenge.
- **OP2: Specification Acquisition.** Our method assumes an enumerable specification surface. For legacy codebases with implicit specifications, automating surface extraction from telemetry, documentation, and user behavior is a critical bottleneck for adoption.
- **OP3: Multi-Objective Drift.** Current drift metrics focus on functional correctness. Extending the framework to simultaneously monitor non-functional requirements (latency, security, fairness) without human intervention remains open. Our drift detection (Section 5.5) would need to compose multiple objective functions without one dominating the pause-gate signal.
- **OP4: Sycophancy at Scale.** The Discussion Manager mitigates sycophancy in 3-model debates (Section 7), but optimal model composition and debate protocols for larger, heterogeneous agent swarms are unknown. Our corpus of 23 discussions is too small to establish whether the observed $SS < 20$ threshold generalizes (cf. Yao et al., 2025 [14]).

13.6 When the Kitchen Loop Should NOT Be Used

The framework is not universally applicable. It should not be applied when: (1) ground truth is weak or unobservable — without a reliable oracle, the verification layer provides false confidence; (2) success criteria are highly subjective (aesthetic quality, UX taste) — the oracle cannot arbitrate matters of judgment; (3) the specification surface is not enumerable, as in exploratory R&D where the goal is discovery rather than convergence; or (4) safety-critical domains lack robust external verification — the oracle’s bounded coverage (Section 4.1) is insufficient when failures have irreversible consequences.

14 The Human-AI Collaboration Model

14.1 Not Either/Or — Both

Chen et al. (2025) identify three automation points — human-only, copilot, and agent — finding agents achieve 60% task correctness vs. 25% for copilots, yet 60% of participants would not continue using agents due to a comprehension gap [13]. The Kitchen Loop operates at a fourth point: **fully autonomous loop with asynchronous human oversight**, eliminating the idle-time and comprehension problems Chen et al. identify. On Shapiro’s five-level autonomy scale [18], this places the Kitchen Loop between L4 (spec-driven development) and the L5 “Dark Factory” — autonomous within each iteration, but with humans still owning the specification surface and strategic direction.

The Kitchen Loop is explicitly a **complementary system**, not a replacement:

Concern	AI (Kitchen Loop)	Human
Coverage	Exhaustive — 1000x the scenarios a human would test	Strategic — focuses on what matters most given user context
Bug discovery	Bottom-up — finds what’s broken by trying everything	Top-down — knows what users are complaining about
Ticket writing	Precise, file-level, with reproduction steps	Context-rich, business-aware
Implementation	Fast, consistent, follows established patterns	Creative, architectural, can break patterns when needed
Code review	Multi-model parallel tribunals	Domain expertise, security intuition
Backlog curation	Deduplication, severity assessment, dependency graphs	Strategic priority, business value, external signals

14.2 The Human’s Highest-Leverage Input

The human’s primary contribution is **specification and backlog curation**: understanding users, monitoring the competitive landscape, and converting those signals into tickets. In our deployments, this required ~30-60 minutes per week once the loop was calibrated — but this figure assumes a mature specification surface (see Section 11.4 for scaling constraints and the human bottleneck at high iteration speeds).

14.3 The External Feasibility Checker

The Kitchen Loop optionally uses an external AI (a different model from the loop’s primary agent) as a **feasibility checker** before committing to an idea.

Empirical results (285+ combined iterations): ~78% PROCEED, ~15% REDIRECT, <5% REJECT + timeout. The low rejection rate suggests the loop’s scenario selection is well-calibrated. REDIRECT cases adjusted scope productively.

15 The Self-Improving Loop

15.1 Meta-Level Improvement

The loop-review skill audits loop behavior every N iterations and produces tickets for infrastructure improvements. Sections 8.4 and 9.3 showed specific examples; the full catalog across both deployments:

- **Platform-specific failures**: Merge automation memory bug on Apple Silicon caused 5 consecutive stalls; loop review identified the pattern and the Execute phase fixed it.
- **Resource contention**: Process collisions when human and loop run concurrently; tool version drift breaking the review phase. Fixes added process tracking and version pinning.
- **Retry and backpressure loops**: PR Manager stuck retrying the same failing PR, wasting entire Polish phases. Fixed by skip-after-2-failures with needs-attention labeling. PR backlog growing unbounded during sustained runs prompted automatic drain mode.
- **State management**: Loop-state lost when worktree PRs weren’t merged promptly (fixed by decoupling state sync from merge lifecycle). Backlog groomer promoting already-in-progress tickets (fixed by counting viable tickets only).

- **Silent failures:** Missing `.env` in worktrees causing strategy failures; loop-review reports silently lost when output directory didn't exist. Fixed by pre-flight checks and write verification guards.
- **Budget management:** Regress timeout consuming 100% of iteration budget, prompting a quick-regression parameter.

The framework has also been applied to its own codebase (dogfooding). In its first iterations running against the KitchenLoop orchestrator and PR manager, the loop discovered and fixed multiple bugs — including race conditions in temporary file handling, missing agent-liveness detection, and incorrect timeout behavior on macOS. This validates that the self-improvement property extends to the meta level: the loop can improve the tool that runs the loop.

15.2 Pattern Consolidation

When the same pattern is confirmed across 2+ iterations, it is promoted from a session observation to a durable memory entry. This creates institutional knowledge that persists across conversations and loop runs.

15.3 The Skill Layer

Starting from 5 basic phase skills, the validated deployments now have 30+ skills covering loop orchestration, quality gates, protocol integration, competitive intelligence, release management, and documentation maintenance. Each skill emerged from the loop's own discovery of a repeatable workflow worth encoding. Additionally, the loop developed a **gate rejection memory system** that prevents redundant LLM auditor calls — if a PR was rejected and no new commits were pushed, it is immediately marked `NOT_MERGEABLE` without wasting an audit cycle. This optimization, discovered and implemented by the loop itself, eliminated the zero-backpressure bottleneck observed in earlier iterations.

Recent additions include `/loop-review-meta` — a macro-level analysis skill that aggregates findings across all loop review reports to surface systemic trends and strategic recommendations.

16 Generalization: Making the Kitchen Loop Portable

16.1 What Makes a Codebase Loop-Ready

The Kitchen Loop works on any codebase where:

1. **The specification is enumerable:** The product has a definable set of features, platforms, and action types that can be expressed as a coverage matrix.
2. **Usage can be automated:** “Using the product” can be performed by an LLM agent without physical interaction (APIs, CLIs, SDKs).
3. **Quality is measurable by regression:** There exists a test oracle that can answer “is the system still working?” in bounded time.

16.2 How to Adapt the Kitchen Loop to Your Domain

Domain	Specification Surface	Regression Oracle	Example Adaptation
Web Application	Pages x User Flows x Browsers	Browser automation + visual regression (Playwright, Cypress)	Ideate generates user journeys; Regress runs screenshot diffing against known-good baselines
ML Pipeline	Models x Datasets x Metrics	W&B / MLflow run comparison + statistical tests	Ideate trains model variants; Regress compares against baseline metrics with significance tests

Domain	Specification Surface	Regression Oracle	Example Adaptation
Smart Contracts	Functions x Chains x Edge Cases	Foundry/Anvil fork execution + invariant checks	Ideate writes fuzz test scenarios; Regress runs invariant test suites
Backend API	Endpoints x Methods x Auth Roles	Contract testing + live traffic shadow comparison	Ideate generates API call sequences; Regress replays against shadow traffic
Mobile App	Screens x Gestures x Devices	Appium automation + visual regression	Ideate scripts user flows; Regress runs on device farm
Compiler / Language	Grammar x Optimizations x Targets	Test suite execution + benchmark comparison	Ideate generates programs exercising edge cases; Regress runs conformance suite

The two adaptation points are always the same: *what is the specification surface?* and *what is the regression oracle?* Everything else — backlog management, phase sequencing, drift control, self-improvement — transfers intact.

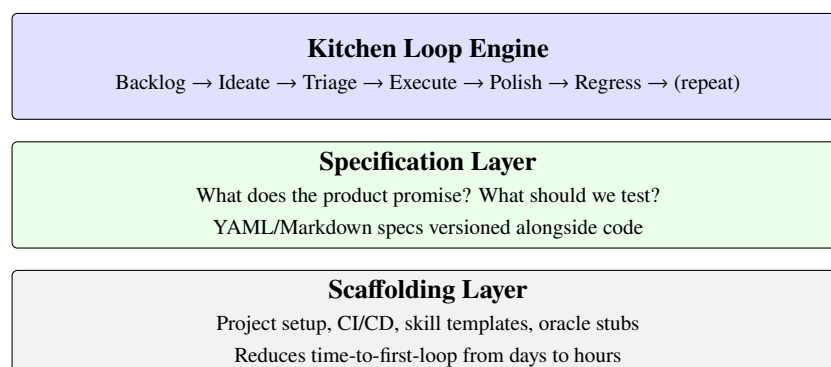
16.3 The Specification Layer

The Kitchen Loop works because the products it's been applied to have well-defined specifications. Most codebases lack this. A **specification layer** — structured YAML or Markdown specs stored alongside code — solves this by making specs machine-readable, version-controlled, and auditable.

16.4 The Scaffolding Layer

A **scaffolding layer** — templates for the six-phase orchestrator, ticket management stubs, oracle skeletons, and CI workflows — reduces time-to-first-loop from days to hours.

16.5 A Composable Stack



17 Conclusion

17.1 The Shift

In our observation, software engineering is undergoing a phase transition. Code production — once the bottleneck — is becoming a commodity. Code review — once a human-only activity — is becoming automated. The competitive advantage is shifting to:

1. **Specification:** Knowing what to build (the AaU1000 method)

2. **Verification:** Proving it works (unbeatable tests)
3. **Convergence:** Ensuring it keeps working (regression and drift control)

17.2 The Evidence

Across two production systems and 285+ combined iterations (Sections 8.3 and 9.3), zero regressions were detected by the regression oracle, quality gates improved monotonically from 76–91% to 100%, and the loop autonomously fixed 17+ infrastructure bugs in its own tooling. The second system validated portability: the same architecture applied to a fundamentally different domain achieved comparable results with a 25x faster iteration cycle, requiring only two reimplemented components (specification surface and regression oracle). The unified trust model held under sustained autonomous operation without increasing human intervention.

17.3 The Invitation

The Kitchen Loop is not the future of software development. It is a practice available today, on a codebase of any size, with tools that already exist. The prerequisites are:

1. **An enumerable specification surface** — what does your product claim to do?
2. **An automatable test environment** — can an AI agent exercise your product?
3. **A regression oracle** — can you answer “is the system still working?” in bounded time?
4. **The discipline to run it continuously** — and trust the output when the tests are unbeatable.

In our experience, the tests are the trust layer. The spec is the compass. The loop is the engine. Given sufficient verification infrastructure, the product evolves itself.

17.4 Toward the Dark Factory

Shapiro (2026) defines a five-level autonomy scale for AI-assisted software development, ranging from “spicy autocomplete” (L1) to the **Dark Factory** (L5) — a regime where specifications go in and working software comes out with no human writing or reviewing code [18]. The Kitchen Loop does not yet achieve L5. Human judgment remains essential for specification design, backlog grooming, and production promotion (Section 13.4). However, within a single iteration — from ideation through implementation, review, and merge — the loop operates autonomously at what Shapiro would classify as L4–L5: specs drive execution, multi-model review replaces human code review, and the regression oracle provides the trust layer that makes unattended operation safe. The contribution of this work is the verification infrastructure — unbeatable tests, drift control, and adversarial UAT gates — that makes each step toward removing the human from the loop a safe one. The Dark Factory is the destination; the Kitchen Loop is the road.

17.5 Testable Hypotheses

Our deployments suggest four empirical hypotheses that subsequent work could validate, replicate, or refute:

- **H1:** Coverage-exhaustion systems (as defined in the regime taxonomy, Section 2.9) discover more user-visible failures per iteration than task-completion systems in partially mature products with >50% specification surface coverage.
- **H2:** Adversarial UAT gates — sealed test cards executed by a fresh evaluator with zero implementation context — reduce false-positive readiness assessments compared with implementer-authored test suites alone.

- **H3:** Tier-weighted scenario selection (Foundation 30% / Composition 50% / Frontier 20%) discovers more bugs per iteration than uniform random selection across the specification surface. The superlinear growth of the Composition tier (Section 3.6) predicts that the advantage increases with product maturity.
- **H4:** In user-facing verification tasks, weak-model evaluation (the least capable model available) is a better proxy for real-user verifiability than strong-model evaluation, because weak models fail on the same ambiguities that trip real users.

These hypotheses are intended to facilitate direct replication, extension, or refutation in future agentic systems research.

17.6 Known Limitations & Future Work

The Kitchen Loop has five structural limitations that bound its applicability:

1. **Single-threaded execution.** The current orchestrator runs one iteration at a time. Parallelization across multiple worktrees is architecturally straightforward but not yet implemented (Section 13.3).
2. **Enumerable specification surface required.** The method assumes the product’s capabilities can be listed as a coverage matrix. Legacy monoliths with implicit specifications require a specification-extraction step (OP2, Section 13.5) before the loop can operate.
3. **Oracle quality is the ceiling.** The loop can only catch what the regression oracle can verify. If the oracle misses a failure mode, the loop is blind to it. Oracle transfer across domains (OP1) remains an open research problem.
4. **Human role persists.** Backlog grooming, specification design, and production promotion require human judgment. At scale, human merge capacity — not AI generation speed — becomes the binding constraint (Section 13.4).
5. **Two-domain validation only.** Our evidence spans two production systems in the DeFi domain. Generalization to other domains (Section 16) is architecturally supported but empirically unvalidated.

A The Coverage Matrix (Strategy Framework Example)

The strategy framework’s intent test coverage matrix illustrates the exhaustive approach. Every cell represents a claim: “Protocol X works on Chain Y for Intent Z.” Empty cells are untested claims.

Protocol	Chain	Swap	LP Open	LP Close	Supply	Borrow	Repay	Withdraw
Aerodrome	Base	P0	P0	P0	-	-	-	-
TraderJoe	Avalanche	P0	P0	P0	-	-	-	-
V2								
Uniswap V3	Ethereum	P1	P1	P1	-	-	-	-
Uniswap V3	Arbitrum	P1	P1	P1	-	-	-	-
Uniswap V3	Base	P1	P1	P1	-	-	-	-
PancakeSwap	BSC	P1	-	-	-	-	-	-
V3								
Aave V3	Ethereum	-	-	-	P0	P0	P0	P0
Aave V3	Arbitrum	-	-	-	P0	P0	P0	P0
Aave V3	Base	-	-	-	P0	P0	P0	P0
Compound	Ethereum	-	-	-	P1	P1	P1	P1
V3								
Morpho	Ethereum	-	-	-	P2	P2	P2	P2
Blue								
Enso	Multi-chain	P3	-	-	-	-	-	-
Curve	Ethereum	P3	-	-	-	-	-	-

P0 = critical path, P1 = breadth, P2 = depth, P3 = completeness

B Representative Strategy Examples

B.1 D.1 Multi-Protocol Composability (Tier 2 — Composition)

A Tier 2 strategy exercised four intent types across two protocols in a single strategy:

```

Step 1: SUPPLY 0.5 WETH as collateral to
        Lending Protocol
        : → 3 TXs (approve + supply +          270K gas
            setCollateral)
Step 2: BORROW 311.20 USDC at 30% LTV
        : → 1 TX (borrow)                      286K gas
Step 3: SWAP 155.60 USDC → 0.0749 WETH via
        DEX
        : → 3 TXs (approve + approve_reset +    213K gas
            swap)
Step 4: LP_OPEN WETH/USDC range [1867--2282]
        : → 4 TXs (approve + approve_reset +    526K gas
            approve + lp_mint)
-----
Total: 11 transactions, 1.3M gas                zero bugs

```

All four verification layers passed on every intent. This was the first test of cross-protocol enrichment data handoff — proving that the *seams* between components work, not just the components themselves.

B.2 D.2 Additional Foundation Examples (Tier 1)

Two Tier 1 scenarios illustrate the “obviously missing” signal. A basic DEX swap discovered that the protocol router configuration had no entry on ANY chain (compilation failed) and used a different interface version (8 vs. 7 parameters, causing silent reverts) — both fixed in the same iteration. Separately,

the first-ever strategy on a new chain discovered a missing native token symbol — a one-line omission that blocked ALL strategies on the entire chain.

C Signal Platform Quality Gate Architecture

C.1 E.1 Verifier Families

Verifier Family	Signal Types Covered	Data Source
Protocol TVL	Discovery, TVL migration, undervalued protocol	DeFi analytics API
Pool Yield	Yield opportunity, LP opportunity, cross-DEX arbitrage	Pool analytics API
Lending Rate	Lending arbitrage, utilization spike	Lending analytics API
Governance	Governance catalyst, proposal tracking	Governance API
Exploit	Exploit detection, exploit warning	Security feeds
Price Feed	Whale alert, depeg risk	Price oracle API
Derivatives	Funding extreme, OI divergence, liquidation cluster	Derivatives aggregator
Emission	Token unlock, emission change	Unlock calendars
Social	Bribe opportunity, sentiment shift, narrative momentum	Social + on-chain
Stablecoin	Stablecoin supply changes	Stablecoin analytics

Coverage: 72/72 signal types (100%) — up from 33/71 at iteration 1. The verifiers are not mocks — they call real APIs and cross-reference signal claims against live data. When The Graph deprecated its free hosted service, the loop detected the failure (agents producing 0 signals), diagnosed the root cause (stale subgraph URLs), and migrated to alternative data sources (GeckoTerminal, DefiLlama) — all autonomously.

C.2 E.2 Anti-Signal Canary Details

Tier 1 — Obviously Bad (6 canaries): - Fabricated event for non-existent entity - Signal with impossible metric value (e.g., 999,999% yield) - False report for an entity that was never affected - Signal with null/empty identifiers - Signal with empty title and description - Signal with out-of-range confidence score

Result: 100% caught across 163 iterations. Zero escapes.

Tier 2 — Shadow (5 canaries): - Real event that already concluded - Known trend reported weeks ago (stale) - Opportunity that dropped below threshold since detection - Valid signal for a deprecated/discontinued entity - Accurate data from a non-authoritative source

Result: Initially 33% caught by L1-L3 (iter 1). Now 100% (iter 163) — improved as verifiers were added and L2 factual checkers gained cross-reference capabilities.

Tier 3 — Adversarial (5 canaries): - Real data + fabricated interpretation - Accurate raw data + incorrect derived conclusion - Valid input data + wrong calculation - Correct methodology applied to wrong time window - Signal mixing real and fabricated sub-claims

Result: Initially 67% caught by L1-L3 (iter 1). Now 100% (iter 163) — the remaining required cross-referencing conclusions against source data, which improved as verifier coverage reached 100%.

Tier 4 — Mixed True/False (5 canaries, added ~iter 100): - Signal with 3 true claims and 2 fabricated claims blended together - Accurate quantitative data with fabricated qualitative assessment - Real protocol event attributed to wrong protocol - Valid historical data presented as current - Correct analysis of real data with one inverted conclusion

Result: 100% caught. Tests the quality gate’s ability to detect partial failures rather than binary pass/fail.

API Degradation Canaries (3 canaries): - Simulated timeout from primary data source - Simulated error response from price oracle - Simulated partial data return (50% of expected fields)

Result: 100% resilience (3/3). Quality gates degrade gracefully — they flag signals as unverifiable rather than producing false passes when dependencies fail.

C.3 E.3 Key Lesson: Rapid Iteration Reveals Intermittent Bugs

One agent’s validation failure rate fluctuated between 75-100% across iterations because its data source API returns different formats at different times. A single test run would show “pass” or “fail” — 163 iterations reveals the ~85% steady-state rate. This class of bug, invisible to traditional CI, demonstrates the value of coverage through repetition, not just breadth.

C.4 E.4 Drift Detection at Scale

The drift detection system monitors quality metrics over a sliding window (5 recent vs. 20 baseline iterations). The key insight: drift detection provides early warning *before* quality gates fail — a 5% drop in factual pass rate over 10 iterations triggers an alert before any individual signal fails hard. This is the mechanism that allows the loop to operate autonomously for 163+ iterations without human supervision.

D Skill Interface Reference

A Kitchen Loop deployment consists of domain-independent orchestration plus domain-specific skills. The following table summarizes the skill interfaces that a new deployment must implement:

Skill	Phase	Input	Output	Domain-Specific?
backlog	Backlog	Ticket state	Promoted tickets	Partially (labels)
ideate	Ideate	Scenario + spec	Report + scenario	Yes (defines “usage”)
triage	Triage	Experience report	Labeled, deduped tickets	Partially (taxonomy)
execute	Execute	Ranked tickets	Branch + PR + tests	Partially (test patterns)
pr-manager	Polish	Open PR list	Reviewed, CI-passing PRs	No
regress	Regress	Codebase state	Pass/fail + drift	Yes (defines “oracle”)
loop-review	Meta	Logs + diffs	Health + improvement	No
review-meta	Meta	All review reports	Trends + recommendations	No

Bold = must be fully reimplemented for each new domain. Non-bold skills transfer with minimal configuration (ticket labels, CI commands, review tool selection).

References

- [1] Fawzy, S., Tahir, A., & Blincoe, K. (2025). Vibe Coding in Practice: Motivations, Challenges, and a Future Outlook — A Grey Literature Review. *arXiv:2510.00328*.
- [2] Huang, Y., Reyna, K., Lerner, B. S., Xia, C. S., & Hempel, J. (2025). Professional Software Developers Don’t Vibe, They Control: AI Agent Use for Coding in 2025. *arXiv:2512.14012*.
- [3] Rahman, M. M., et al. (2026). Task-Level Evaluation of AI-Generated Pull Requests in Open-Source Software. *arXiv preprint*.
- [4] Robbes, R., Matricon, T., Degueule, T., Hora, A., & Zacchiroli, S. (2026). Agentic Much? Adoption of Coding Agents on GitHub. *arXiv:2601.18341*.

- [5] Abtahi, S. & Azim, A. (2025). Augmenting Large Language Models with Static Code Analysis for Automated Code Quality Improvements. *arXiv:2506.10330*.
- [6] Liang, X., Garg, S., & Zilouchian Moghaddam, R. (2025). The SWE-Bench Illusion: When State-of-the-Art LLMs Remember Instead of Reason. *arXiv:2506.12286*.
- [7] Thai, T. D., et al. (2025). SWE-EVO: Benchmarking Coding Agents in Long-Horizon Software Evolution Scenarios. *arXiv:2512.18470*.
- [8] Cai, Y., Li, R., Liang, P., Shahin, M., & Li, Z. (2025). Designing LLM-based Multi-Agent Systems for Software Engineering Tasks: Quality Attributes, Design Patterns and Rationale. *ACM Trans. Softw. Eng. Methodol.* *arXiv:2511.08475*.
- [9] Shukla, S., Joshi, S., & Syed, T. (2025). Security Degradation in Iterative AI Code Generation — A Systematic Analysis of the Paradox. *arXiv:2506.11022*.
- [10] Gao, Y., et al. (2025). A Survey of Bugs in AI-Generated Code. *arXiv:2512.05239*.
- [11] He, H., Miller, C., Agarwal, S., Kastner, C., & Vasilescu, B. (2025). Speed at the Cost of Quality: How Cursor AI Increases Short-Term Velocity and Long-Term Complexity in Open-Source Projects. *Proc. MSR 2026*. *arXiv:2511.04427*.
- [12] Becker, K., Rush, A. M., Barnes, C., & Rein, D. (2025). Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. *METR*. *arXiv:2507.09089*.
- [13] Chen, Y., Talwalkar, A., Brennan, G., & Neubig, G. (2025). Code with Me or for Me? How Increasing AI Automation Transforms Developer Workflows. *arXiv:2507.08149*.
- [14] Yao, S., et al. (2025). Sycophancy in Multi-Agent Debate. *arXiv:2509.23055*.
- [15] Tao, Z., et al. (2025). SICA: A Self-Improving Coding Agent. *arXiv:2504.15228*.
- [16] Novikov, A., et al. (2025). AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery. *DeepMind Technical Report*.
- [17] Confucius SDK (2025). A Meta-Agent Build-Test-Improve Loop for SDK Development.
- [18] Shapiro, D. (2026). The Five Levels: from Spicy Autocomplete to the Dark Factory. *Blog post*, January 23, 2026. <https://www.danshapiro.com/blog/2026/01/the-five-levels-from-spicy-autocomplete-to-the-software-factory/>

E How to Cite

```
@misc{roy2026kitchenloop,
  title      = {The Kitchen Loop: User-Spec-Driven Development
                for a Self-Evolving Codebase},
  author     = {Roy, Yannick},
  year       = {2026},
  eprint     = {2603.25697},
  archivePrefix = {arXiv},
  primaryClass = {cs.SE},
  url        = {https://arxiv.org/abs/2603.25697}
}
```